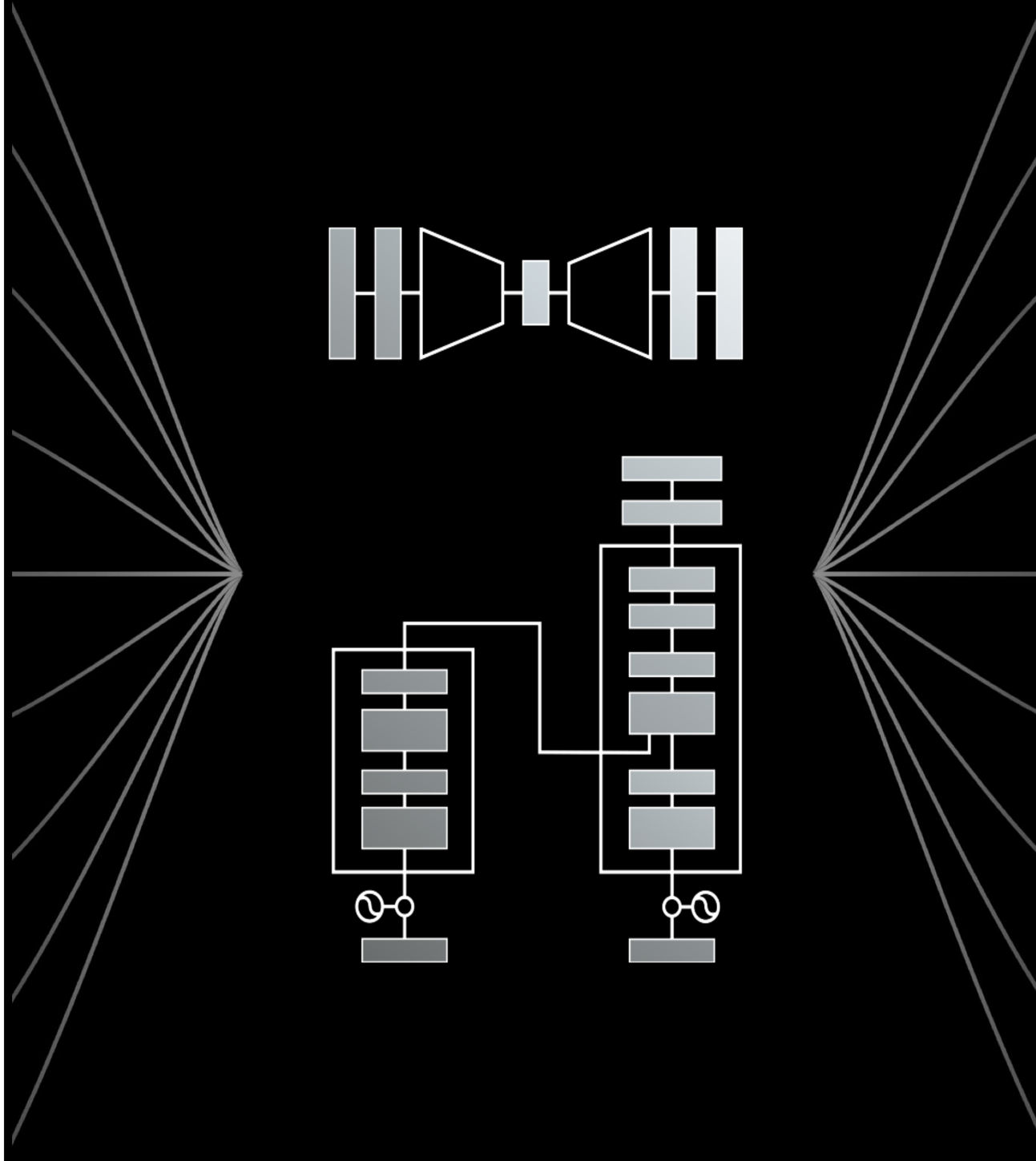
NCHC LLM Bootcamp

Cliff Chiu, Solution Architect | June 17, 2025

- 09:00 AM - 09:10 AM: Welcome Event Overview
- 09:10 AM - 09:20 AM: Introduction to Joint Lab program with NVAITC
- 09:20 AM - 10:20 AM: Introduction to Generative AI and LLMs (Lecture)
- 10:20 AM - 10:30 AM: Break
- 10:30 AM - 11:30 AM: LLM Training and Inference Theory (Lecture)
- 11:30 AM - 11:40 AM: Break
- 11:40 AM - 13:00 AM: Demo: NeMo, NIM

課程目標

- **瞭解 LLM 的運作方式**
  - 運作原理
  - 訓練過程
- **學習大型語言模型的訓練與推論**
  - 訓練大型語言模型的挑戰
  - 高效的模型推論技巧
- **瞭解部署大型語言模型伺服器**



大型語言模型簡介

什麼是生成式AI (Generative AI)？

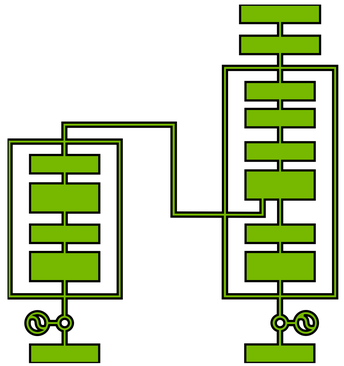
AI模型：A轉換成B

AI模型

A



轉換



轉換

B



什麼是生成式AI (Generative AI)？

轉換各種類型的資料來產生新的內容

A

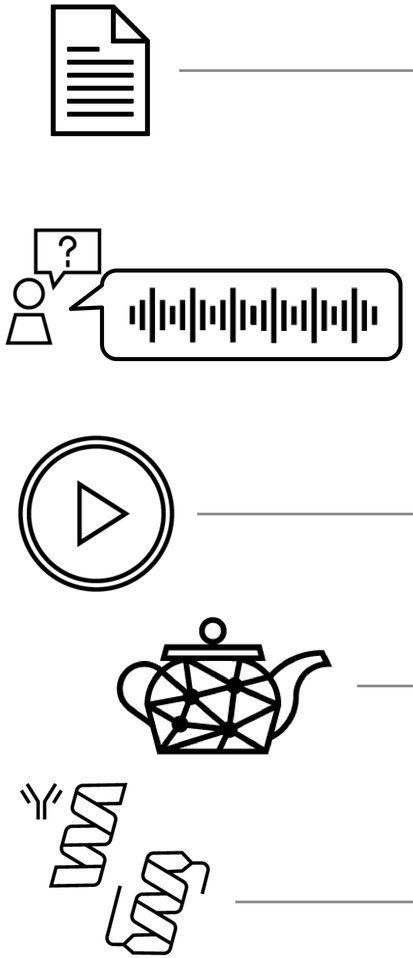
文字

聲音

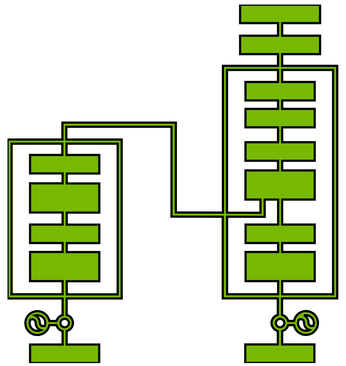
影像

3D物件

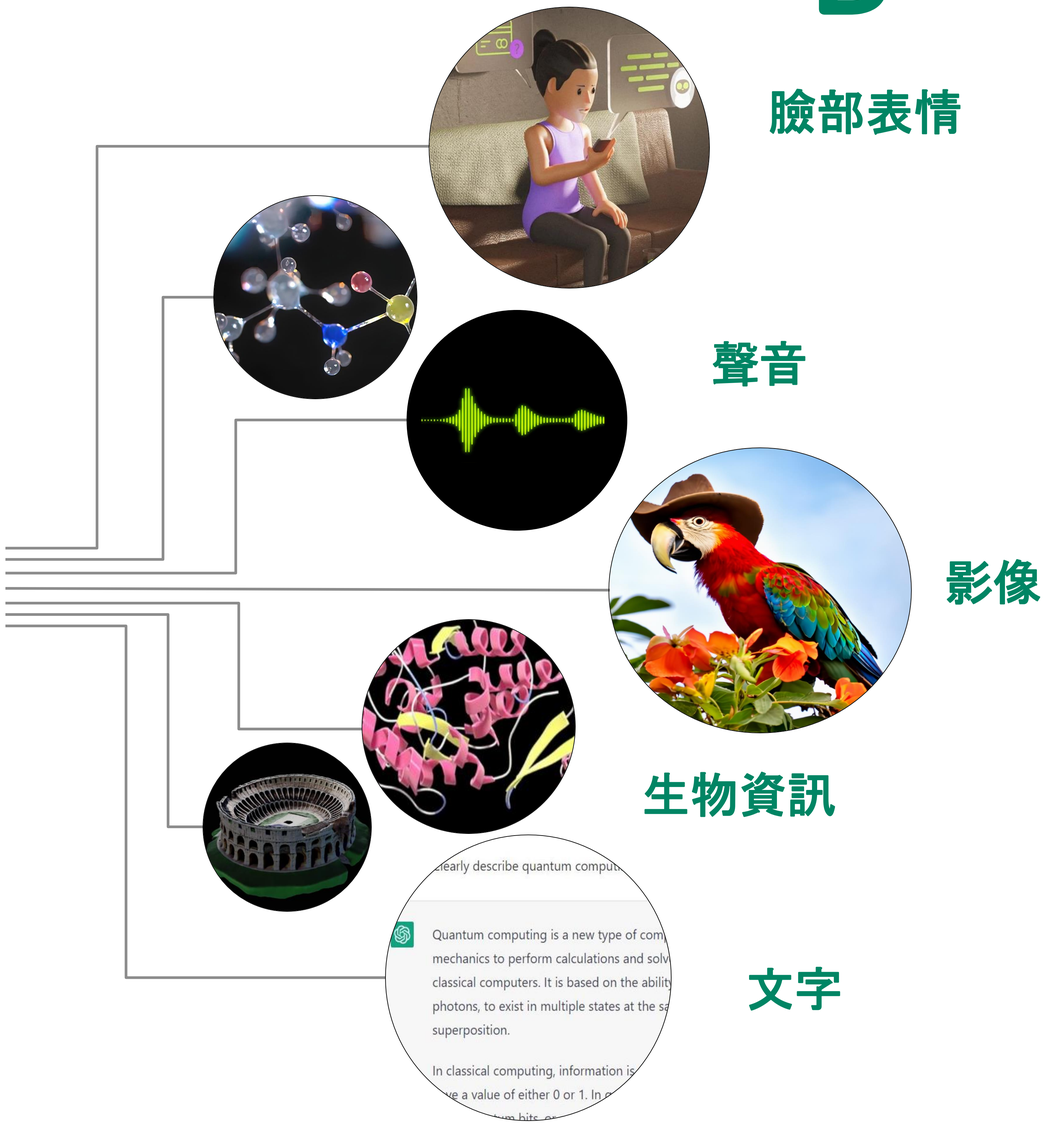
生物資訊
(DNA)



生成式AI模型



B



臉部表情

聲音

影像

生物資訊

文字

NVIDIA

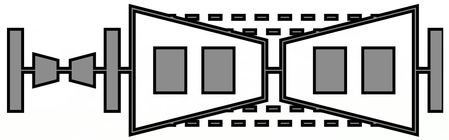
Generative AI vs Discriminative AI

生成式AI vs 判別式AI

輸入



判別式AI

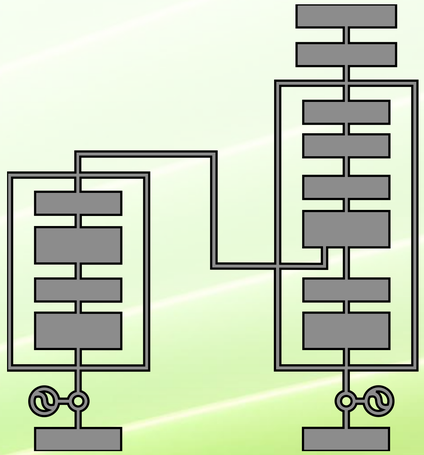


判斷類別 （選擇題）

0.2	0.9	0.6	0.1
狗	貓	虎	人

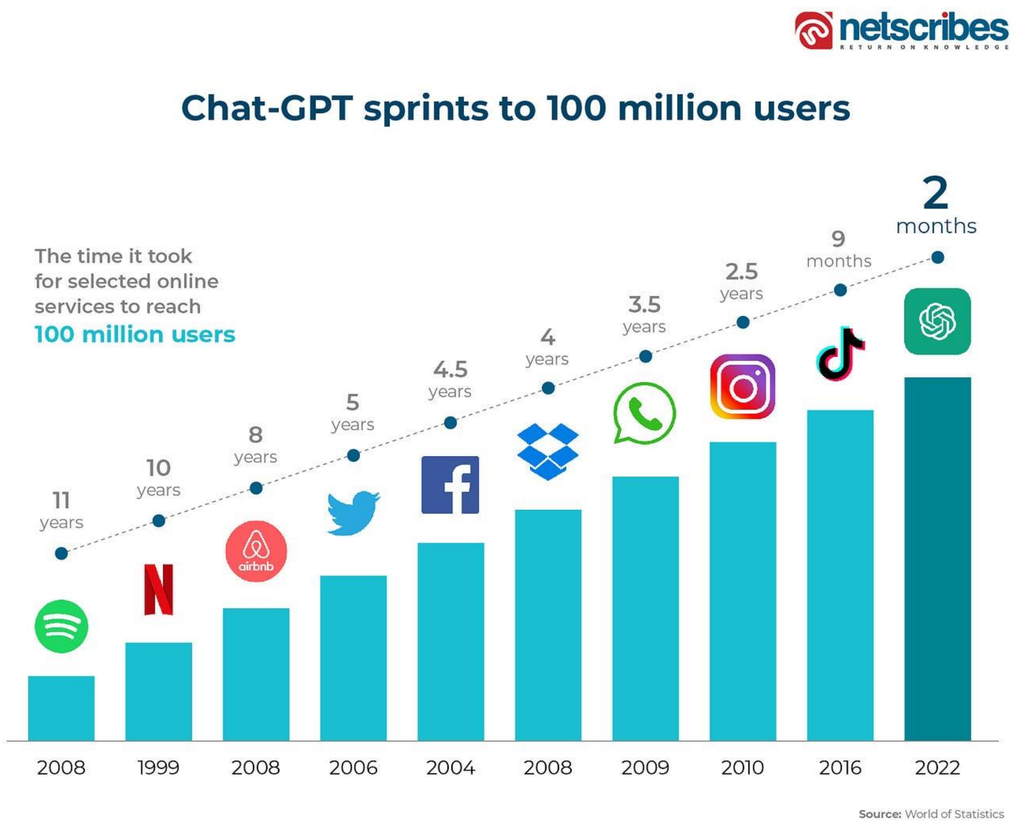
生成式AI

請你幫我畫一張可愛的貓咪照片，貓咪的臉上噴灑了許多不同的顏色背景中有彩色的煙霧。



產生具有真實意義的內容（**作文題、問答題**）





生成式AI模型可以哪些事情？

寫文案、產生會議紀錄



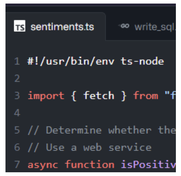
- Summarization
- GPT-4
- Marketing Copy

文字翻譯、文法修正



- Translating Wikipedia
- NLLB-200
- Real-Time Translation

寫程式、分析問題



- Dynamic Code Commenting
- Code Llama
- Function Generation

生成影像、3D物件



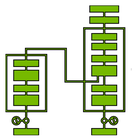
- Brand Creation
- e-Diffi
- Gaming Characters

NVIDIA

ChatGPT是怎麼運作的？

文字接龍



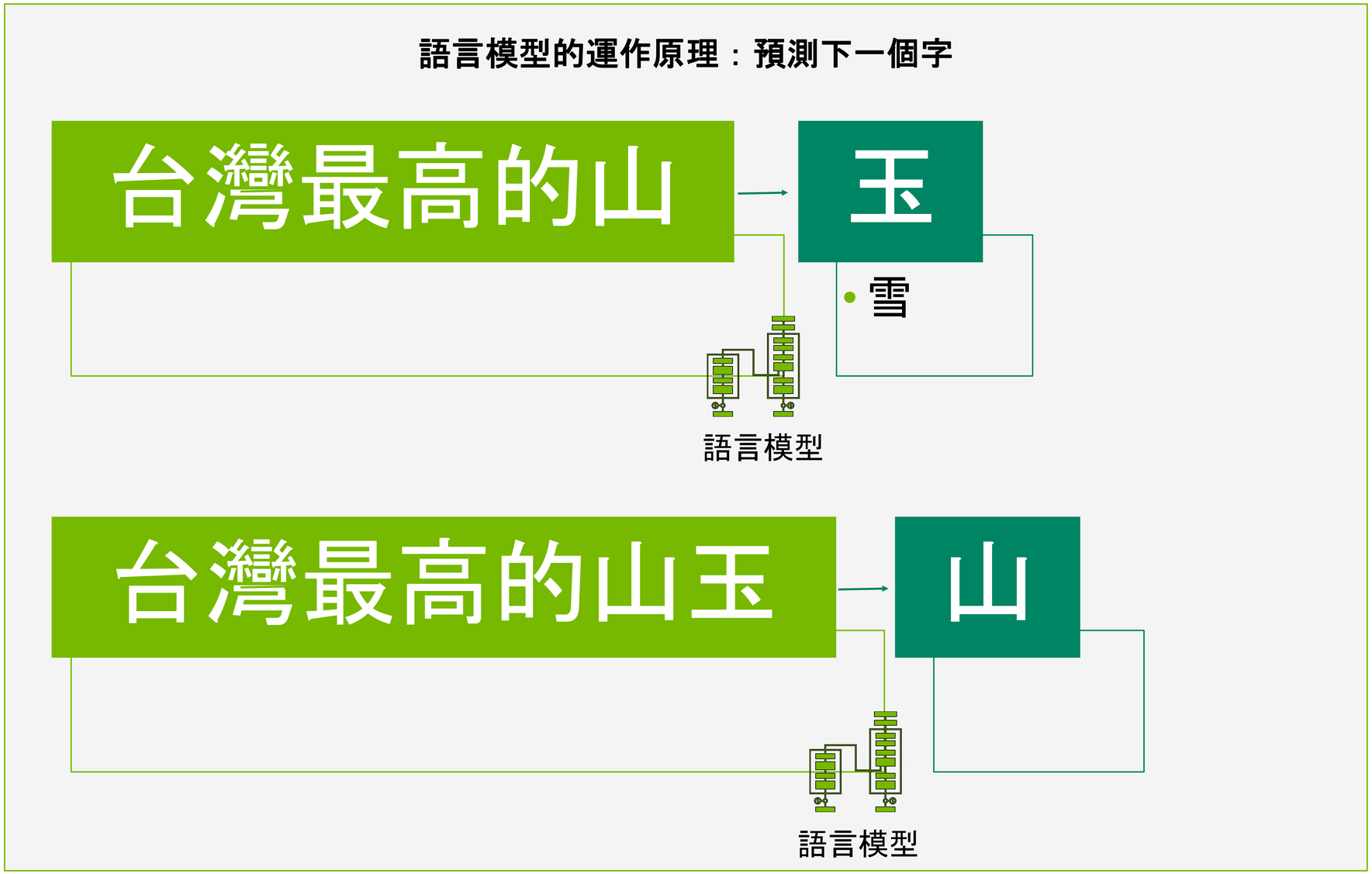


語言模型經由**大量**、各種不同**人類語言**的文本訓練而成

NVIDIA.

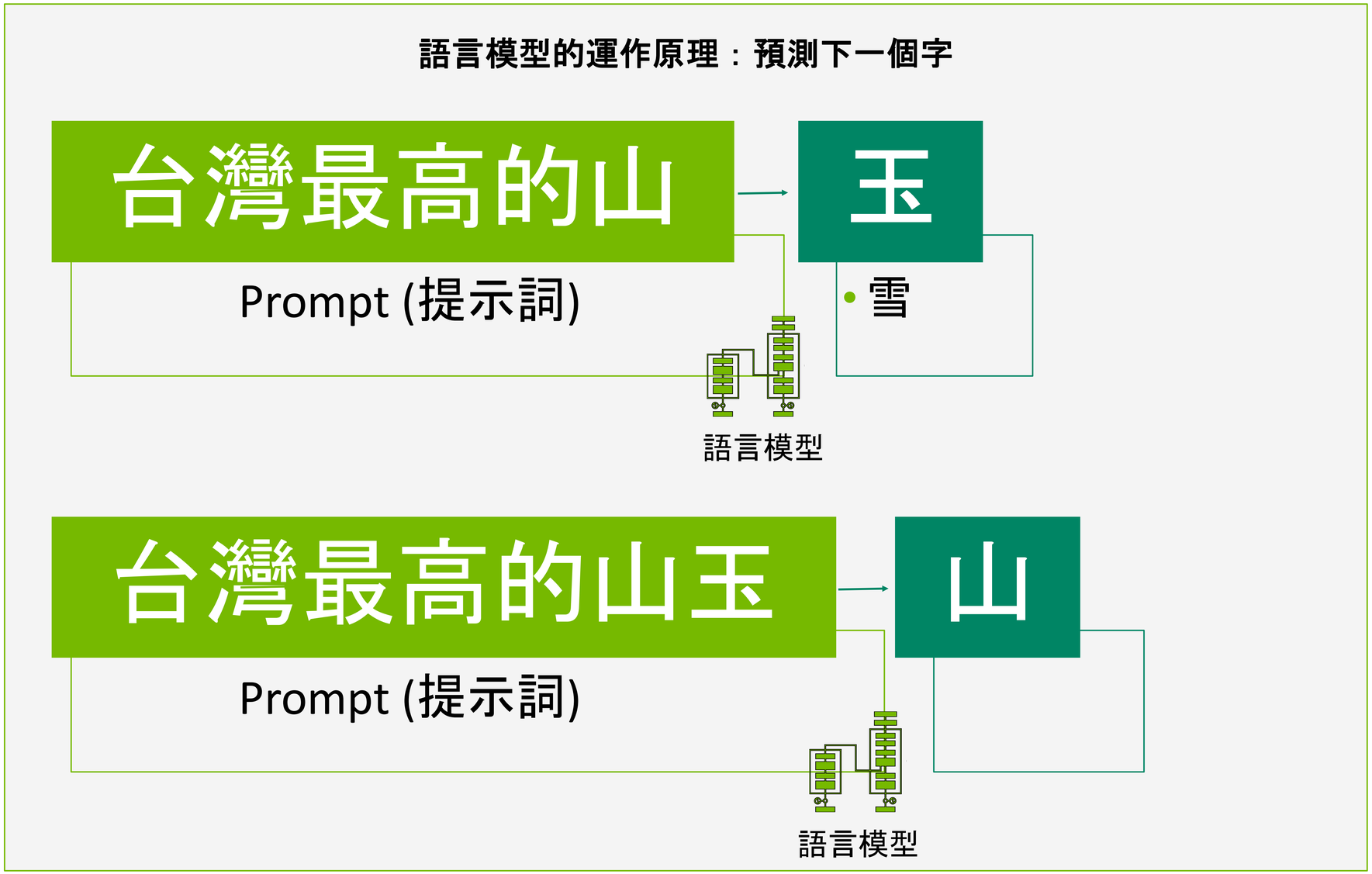
語言模型是怎麼回答問題的？

文字接龍



語言模型是怎麼回答問題的？

文字接龍

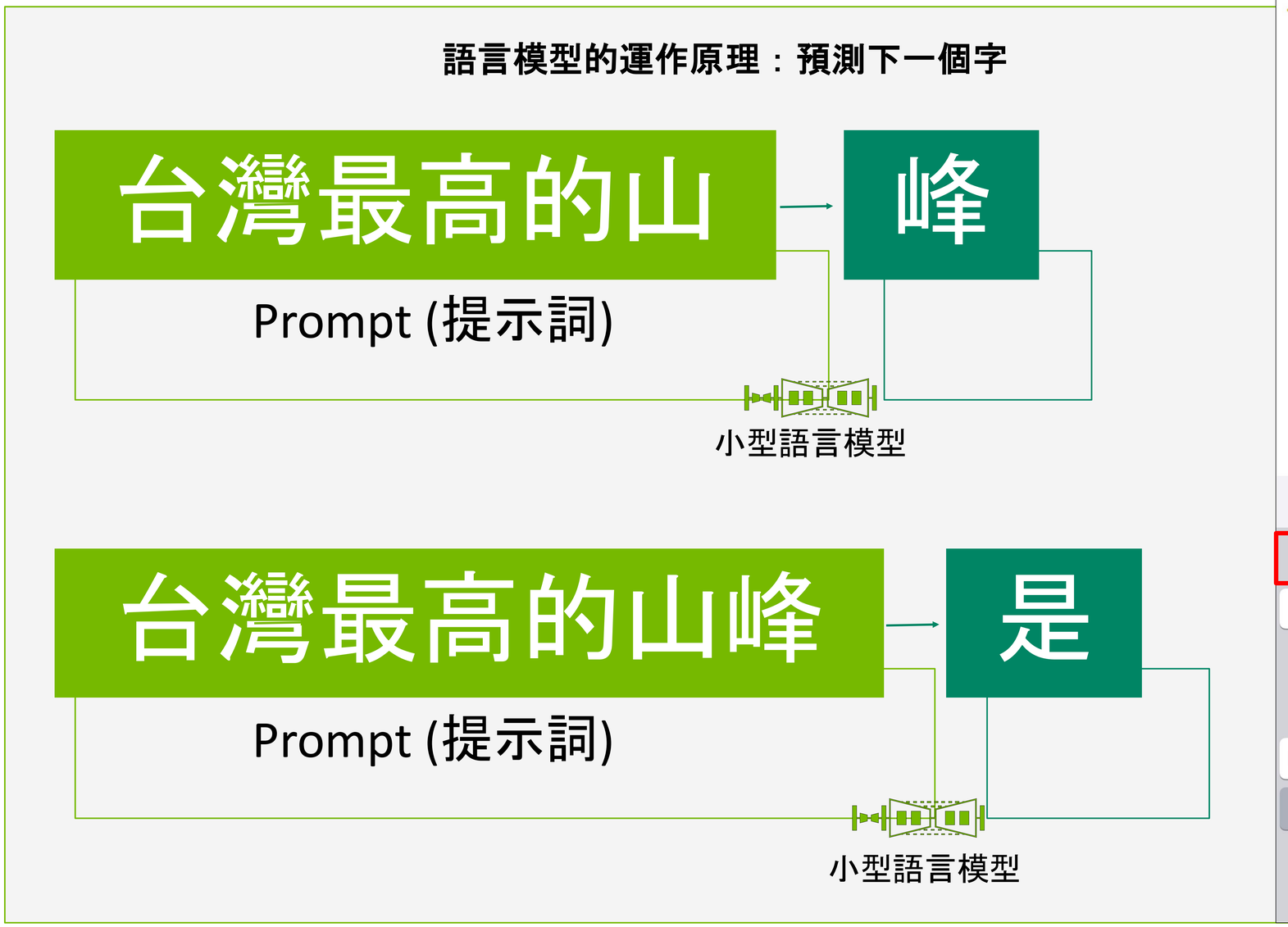


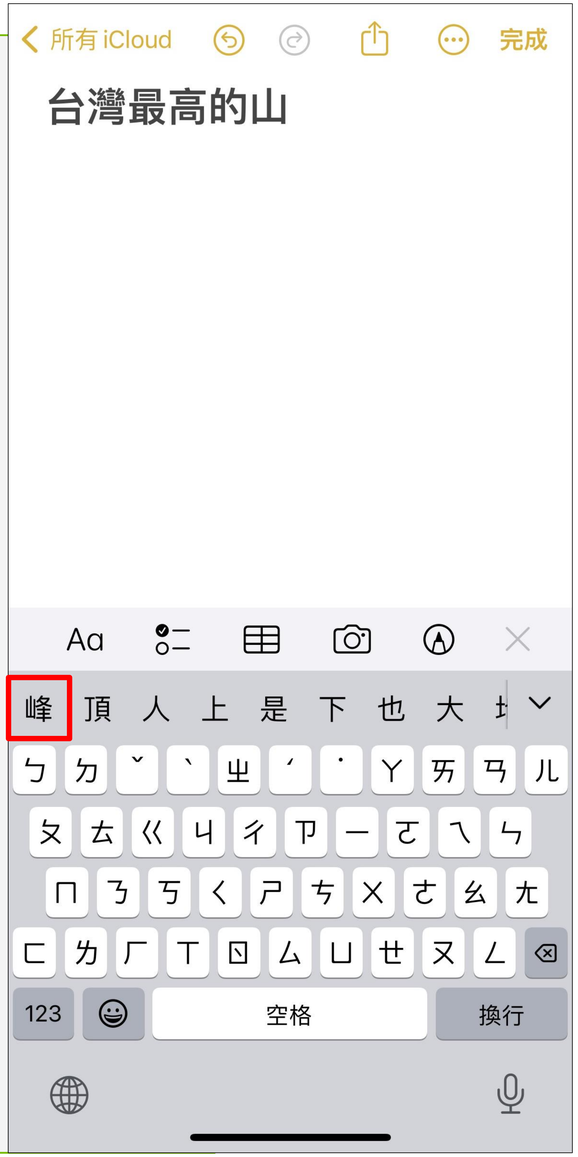
手機輸入法聯想詞

文字接龍

問題 ⇨

問題 ⇨





什麼是大型語言模型?

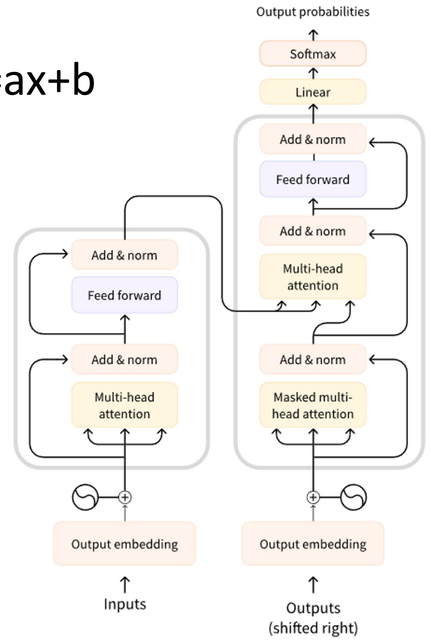
一種複雜的深度學習模型

- 大型語言模型是一種有**大量參數**，並以**巨量文本**訓練而成的深度學習模型，能學習語言結構、語義及上下文，廣泛應用於自然語言處理，如語言生成、翻譯、摘要等，提高文本處理的效率和準確性。

大量的模型參數：先天的腦容量　　**巨量的文本資料**：後天閱讀的知識量

F(x)=ax+b

2個參數



Transformer

模型名稱	參數量	預訓練詞元數	GPU 小時
GPT-3	175B	500 B tokens	836 K
Phi-4	14B	9.8 T tokens	967 K
Llama3	8B	15 T tokens	1.46 M
	70B	15 T tokens	7 M
	405B	15 T tokens	30.84 M

研究人員發現**越大的模型**與**越多的資料**對語言模型的表現越好。
使得大型語言模型能夠更精準學習複雜的語言模式。

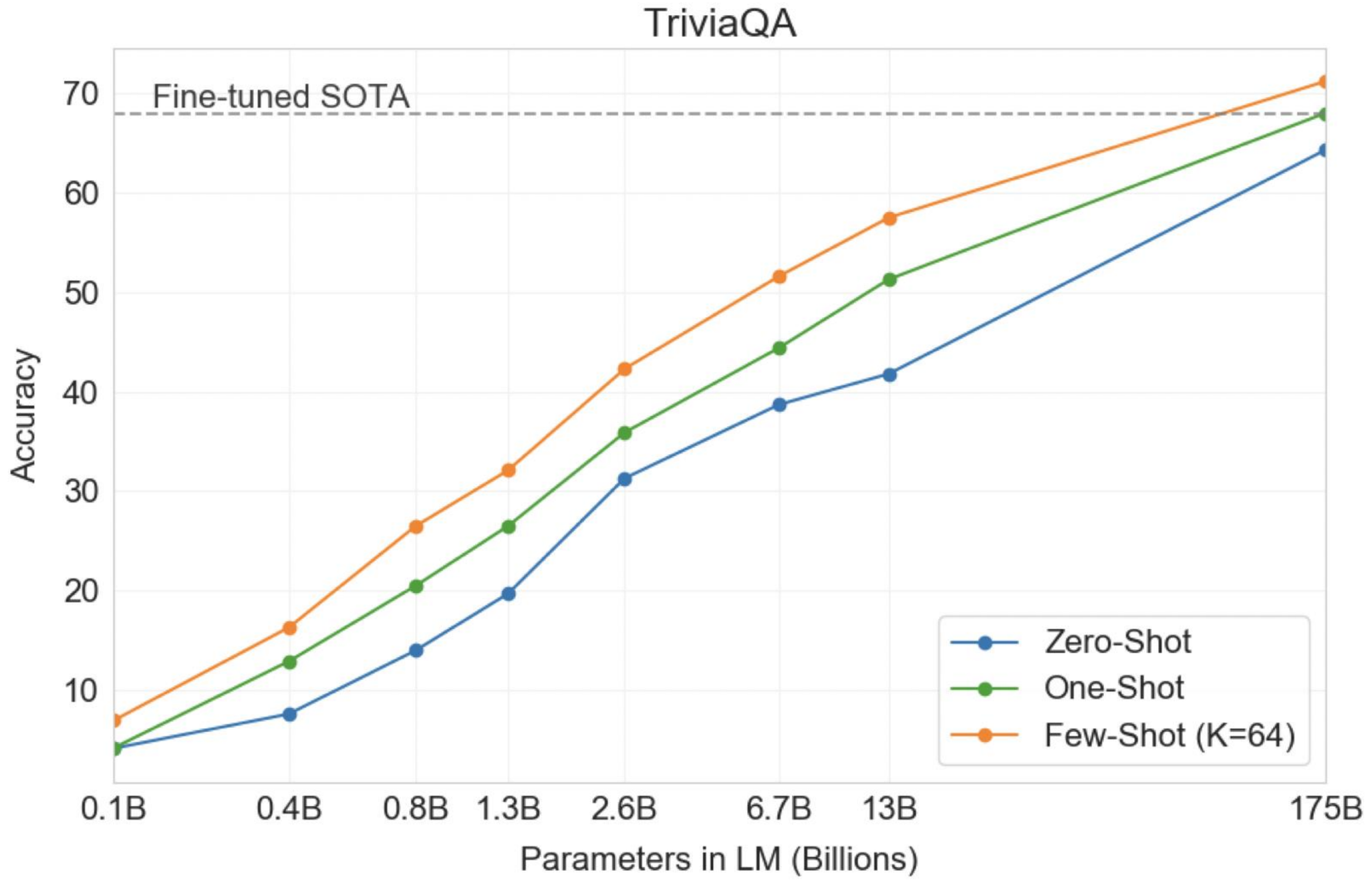
NVIDIA

On TriviaQA, GPT-3's performance

當語言模型參數量大到效果足以媲美專為特定任務微調的小模型

研究人員發現**越大的模型**與**越多的資料**對語言模型的表現越好。

使得大型語言模型能夠更精準學習複雜的語言模式。



大型語言模型到底看過多少的資料？

以哈利波特全集作為基準

哈利波特全集



200 萬個字元

V.S

大型語言模型訓練資料集



15 兆個字元以上

➡ 至少看了800萬遍的哈利波利全集

NVIDIA

大型語言模型的訓練過程

訓練方法	模型	基礎建設

第 1 階段
學習大量知識

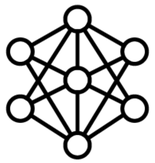


Pre-training 預訓練

- GPT-3
- Llama Base
- Nemotron Base

GPU SuperPods





Foundation Model 基礎模型



第 2 階段
學習問答能力

Fine-tuning 微調

- ChatGPT
- Llama Instruct
- Mistral Instruct
- Nemotron

GPU Nodes

- 領域適應預訓練 Domain-Adaptive Pre-training
- 持續預訓練 Continuous Pre-training

- 全參數微調 Supervise Fine-tuning
- 高效參數微調 Parameter Efficient Fine-tuning

- 模型對齊 Model Alignment - RLHF/DPO/GRPO



大型語言模型



NVIDIA.

大型語言模型的訓練過程

	訓練方法	模型	基礎建設
第 1 階段 學習大量知識	Pre-training 預訓練	• GPT-3 • Llama Base • Nemotron Base	GPU SuperPods

訓練資料

原始語料：世界第一高峰是喜馬拉雅山

非監督式學習

世界第一高峰是 → **語言模型** → 喜馬拉雅山

大型語言模型的訓練過程

訓練方法	模型	基礎建設

第 1 階段
學習大量知識

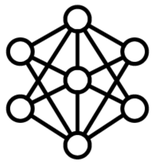


Pre-training 預訓練

- GPT-3
- Llama Base
- Nemotron Base

GPU SuperPods





Foundation Model 基礎模型



第 2 階段
學習問答能力

Fine-tuning 微調

- ChatGPT
- Llama Instruct
- Mistral Instruct
- Nemotron

GPU Nodes

- 領域適應預訓練 Domain-Adaptive Pre-training
- 持續預訓練 Continuous Pre-training

- 全參數微調 Supervise Fine-tuning
- 高效參數微調 Parameter Efficient Fine-tuning

- 模型對齊 Model Alignment - RLHF/DPO/GRPO



大型語言模型

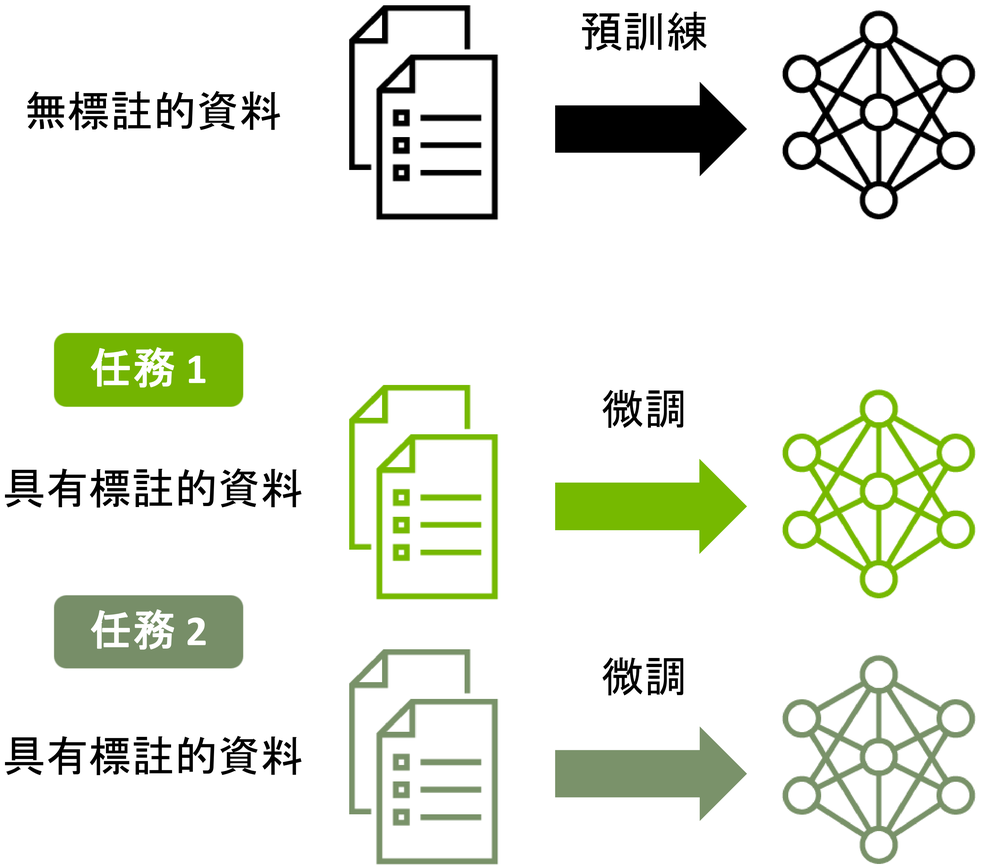


NVIDIA.

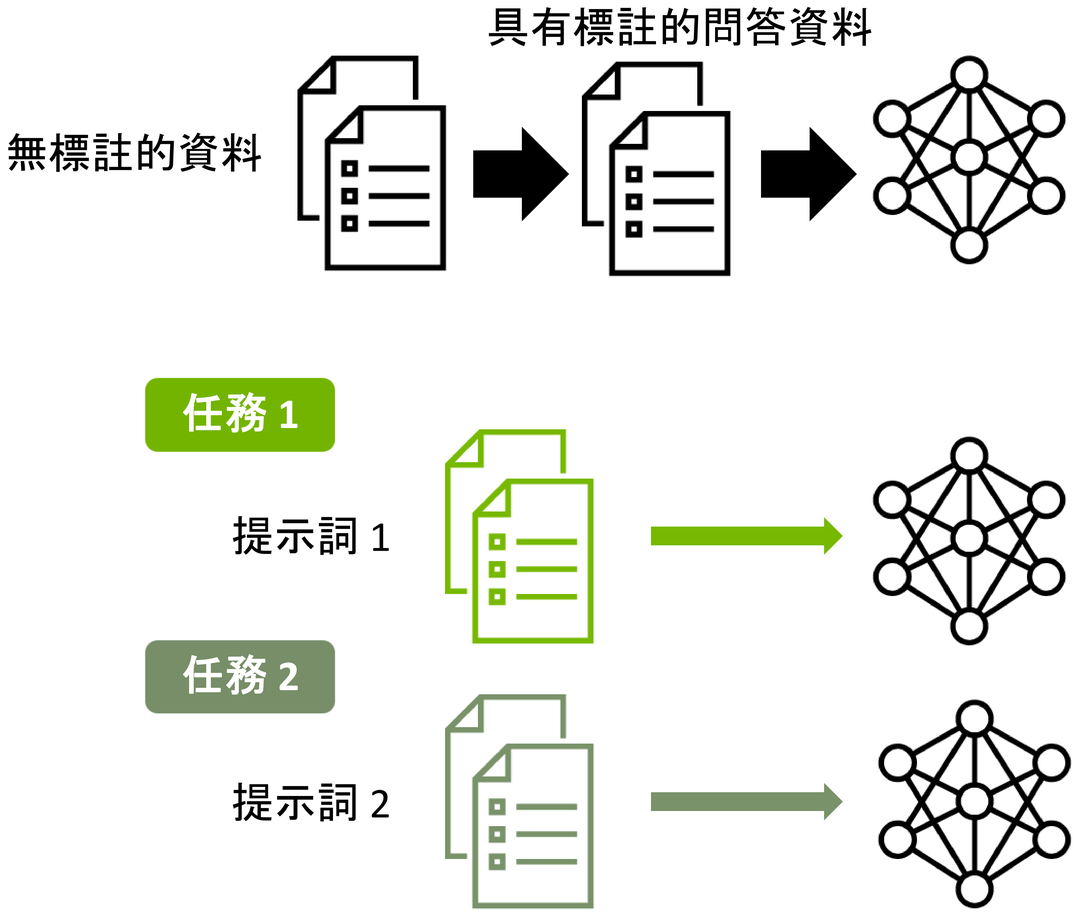
語言模型典範轉移

特定任務微調 vs 指令微調

預訓練 + 特定任務微調

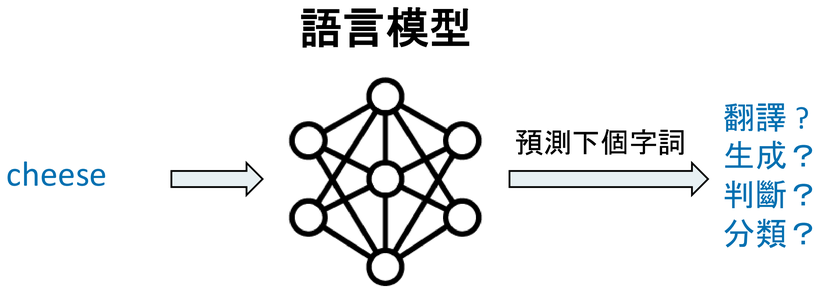


預訓練 + 指令微調

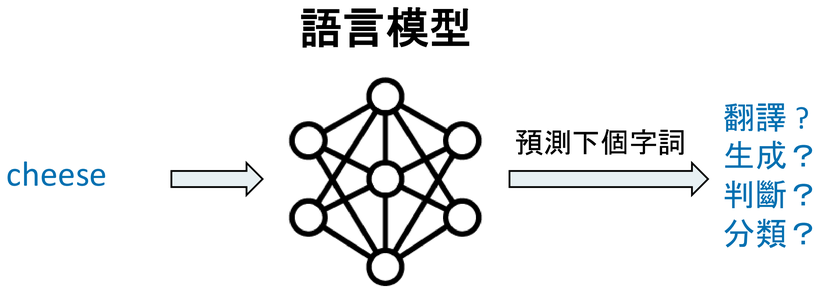


NVIDIA

In-context Learning



In-context Learning



Zero-shot

將英文翻譯成中文：
cheese -> _____

任務描述
提示 prompt

One-shot

將英文翻譯成中文：
sea otter -> 海獺
cheese -> _____

任務描述
範例
提示 prompt

Few-shot

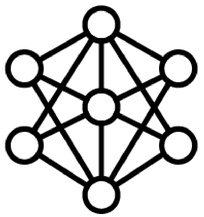
將英文翻譯成中文：
sea otter -> 海獺
peppermint -> 薄荷
plush giraffe -> 絨毛長頸鹿
cheese -> _____

任務描述
範例
範例
範例
提示 prompt

In-context Learning

語言模型

cheese



預測下個字詞

翻譯？
生成？
判斷？
分類？

Zero-shot

將英文翻譯成中文：　← 任務描述

cheese -> _____　← 提示 prompt

One-shot

將英文翻譯成中文：　← 任務描述

sea otter -> **金魚**　← **範例**

cheese -> **起司**　← 提示 prompt

Few-shot

將英文翻譯成中文：　← 任務描述

sea otter -> **金魚**　← **錯誤範例**

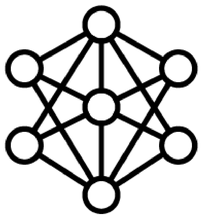
peppermint -> 薄荷　← 範例

plush giraffe -> **絨毛袋鼠**　← **錯誤範例**

cheese -> **起司**　← 提示 prompt

In-context Learning

語言模型



cheese → 預測下個字詞 → 翻譯？ 生成？ 判斷？ 分類？

Zero-shot

將英文翻譯成中文：　← 任務描述

cheese -> _____　← 提示 prompt

One-shot

將英文翻譯成中文：　← 任務描述

sea otter -> **金魚**　← **範例**

cheese -> **起司**　← 提示 prompt

Few-shot

將英文翻譯成中文：　← 任務描述

sea otter -> **金魚**　← **錯誤範例**

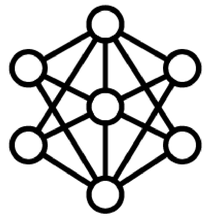
peppermint -> 薄荷　← 範例

plush giraffe -> **絨毛袋鼠**　← **錯誤範例**

cheese -> **起司**　← 提示 prompt

語言模型

將英文翻譯成中文：
cheese ->



預測下個字詞 → 起司

模型本來就會做特定任務
只是需要特定輸入來啟動

指令微調 Instruction Tuning

對齊人類偏好 Alignment

Prompt: *Explain the moon landing to a 6 year old in a few sentences.*

GPT-3

Explain the theory of gravity to a 6 year old.

Explain the theory of relativity to a 6 year old in a few sentences.

Explain the big bang theory to a 6 year old.

Explain evolution to a 6 year old.

照樣造句

指令微調 Instruction Tuning

對齊人類偏好 Alignment

Prompt: *Explain the moon landing to a 6 year old in a few sentences.*

GPT-3

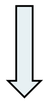
Explain the theory of gravity to a 6 year old.

Explain the theory of relativity to a 6 year old in a few sentences.

Explain the big bang theory to a 6 year old.

Explain evolution to a 6 year old.

照樣造句



Instruction Tuning

InstructGPT

People went to the moon, and they took pictures of what they saw, and sent them back to the earth so we could all see them.

理解問題
完成回答

NVIDIA.

指令微調 Instruction Tuning

對齊人類偏好 Alignment

Prompt: *Explain the moon landing to a 6 year old in a few sentences.*

GPT-3

Explain the theory of gravity to a 6 year old.

Explain the theory of relativity to a 6 year old in a few sentences.

Explain the big bang theory to a 6 year old.

Explain evolution to a 6 year old.

照樣造句

Instruction Tuning

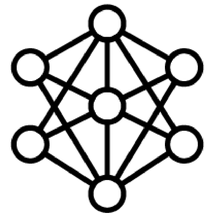
InstructGPT

People went to the moon, and they took pictures of what they saw, and sent them back to the earth so we could all see them.

理解問題
完成回答

Supervised Fine-tuning
理解人類的問題、指令

大型語言模型



RLHF
符合人類偏好的回答

大型語言模型的訓練過程

訓練方法	模型	基礎建設

第 1 階段
學習大量知識

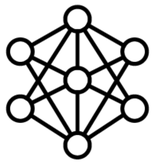


Pre-training 預訓練

- GPT-3
- Llama Base
- Nemotron Base

GPU SuperPods





Foundation Model 基礎模型



第 2 階段
學習問答能力

Fine-tuning 微調

- ChatGPT
- Llama Instruct
- Mistral Instruct
- Nemotron

GPU Nodes

- 領域適應預訓練 Domain-Adaptive Pre-training
- 持續預訓練 Continuous Pre-training

- 全參數微調 Supervise Fine-tuning
- 高效參數微調 Parameter Efficient Fine-tuning

- 模型對齊 Model Alignment - RLHF/DPO/GRPO



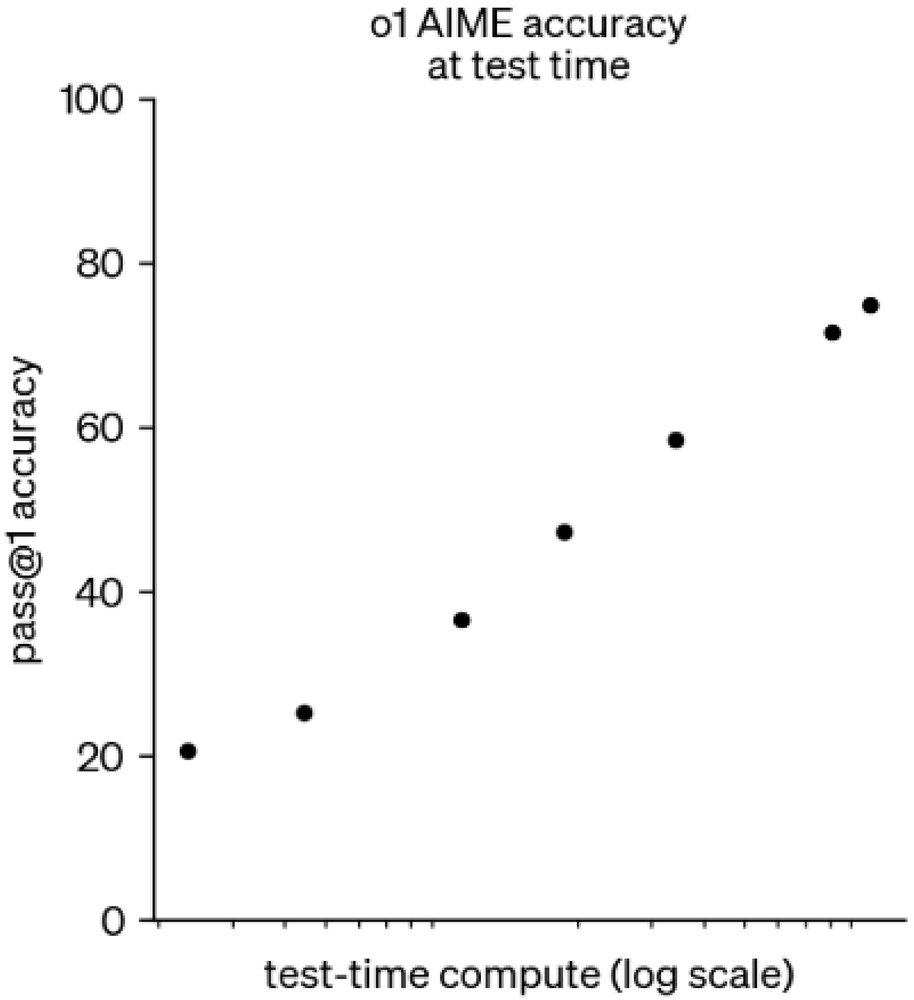
大型語言模型

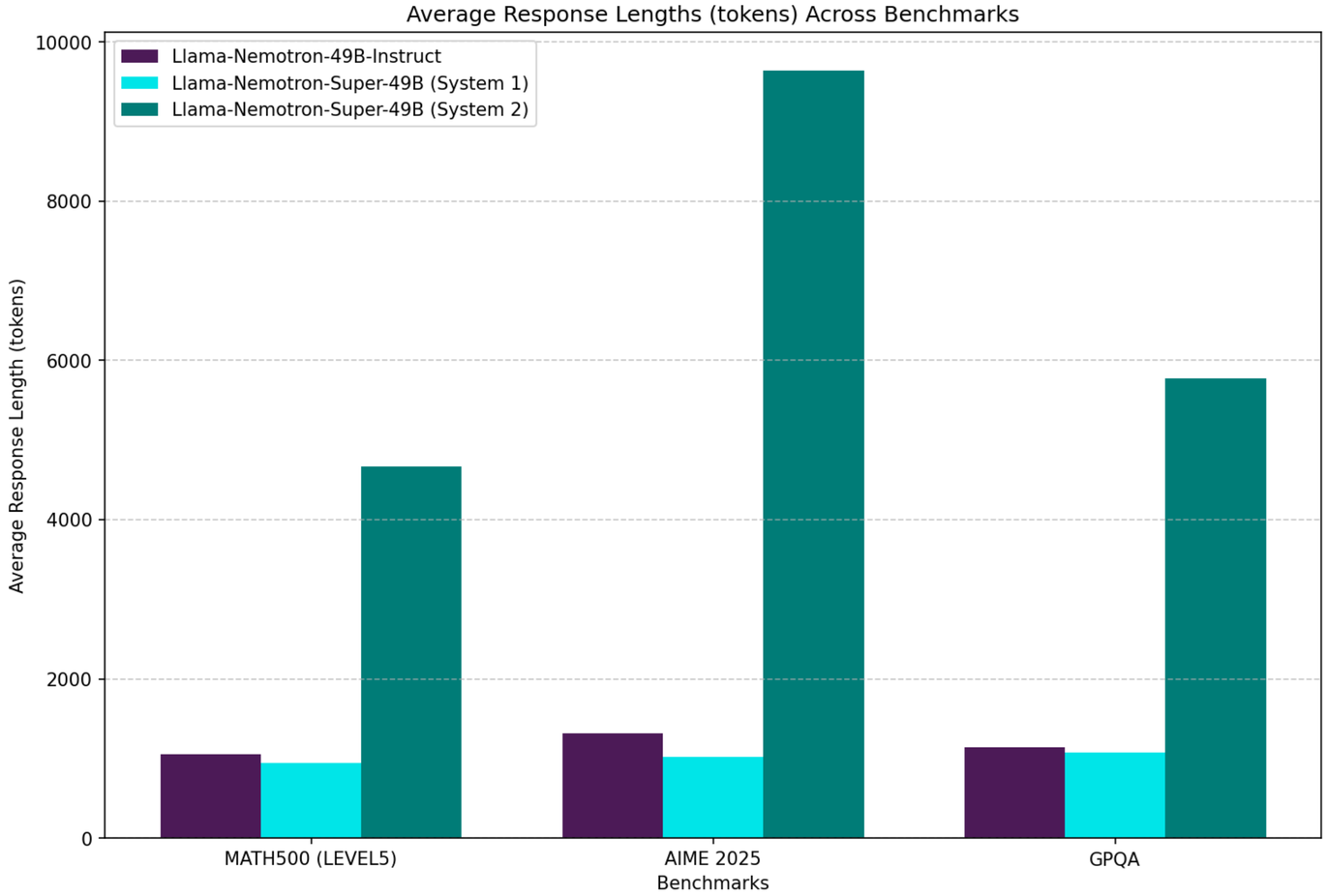


NVIDIA.

什麼是推理？

推理是模型在測試時能夠有效利用更多計算資源以得出更佳答案的能力。

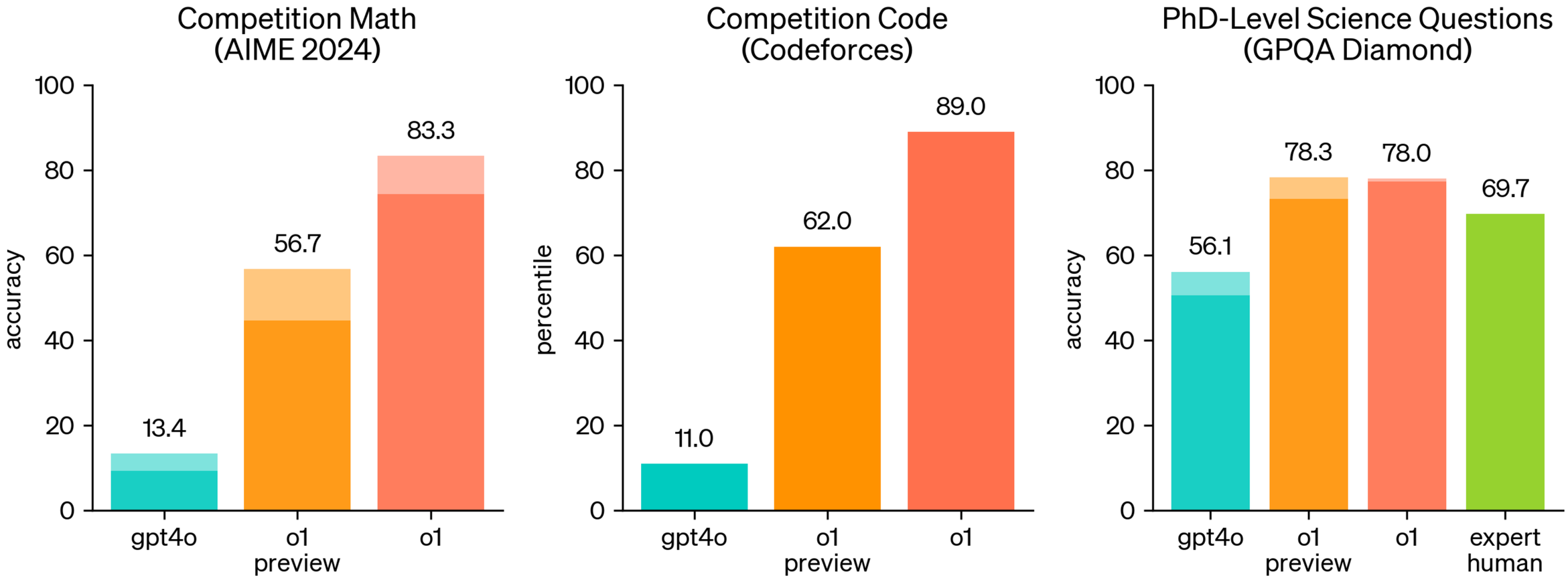




(OpenAI) "Learning to reason with LLMs" 2024 https://openai.com/index/learning-to-reason-with-llms/

什麼是推理？

推理是模型在測試時能夠有效利用更多計算資源以得出更佳答案的能力。



(OpenAI) "Learning to reason with LLMs" 2024 https://openai.com/index/learning-to-reason-with-llms/

比較有無推理能力 (Reasoning ON/OFF)

Example: "Write an efficient Python function to check if the number is both prime and divisible by 5"

ON

Final answer is correct and efficient. But takes tokens to generate

```
<think>
Okay, I need to write a Python function that checks if a number is both prime and
divisible by 5. Hmm, let's think about the requirements here.

First, the number has to be prime. A prime number is a number greater than 1 that has no
divisors other than 1 and itself. Then, it also needs to be divisible by 5. So, the
number must be divisible by 5 and also prime.

Wait a minute.
...

So, the only number that satisfies both conditions is 5. Therefore, the function should
return True only when the input is 5, and False otherwise.

So the steps for the function would be:

...

But wait, what if someone passes a non-integer, like a float? But the problem says
"number", which could be a float. But primes are defined for integers. So perhaps the
function should first check if the input is an integer. Because 5.0 is equal to 5, but
it's a float. So maybe we need to check if n is an integer and equals 5.

...

So, that's the efficient way. Because checking for primality for numbers other than 5 is
unnecessary. Because any other number divisible by 5 is either 5 (prime) or a multiple
of 5 (not prime). So, the function can be optimized to just check if the number is 5.

Therefore, the code is correct and efficient.
</think>

```python
def check_prime_and_divisible_by_five(n):
    return n == 5 and isinstance(n, int)
```
```

OFF

Correct by not efficient/best answer

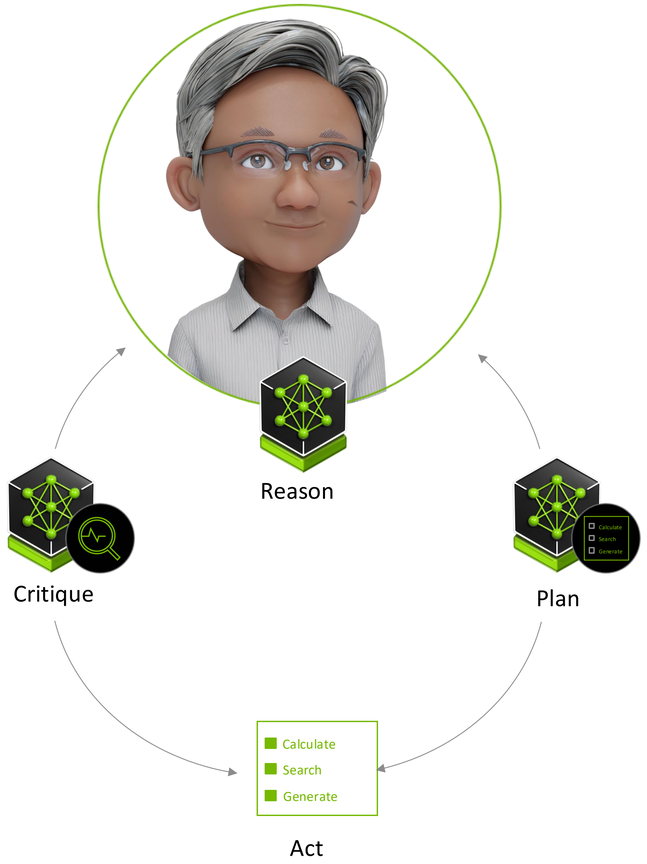
```python
def is_prime(n):
    """Check if a number is prime."""
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    max_divisor = int(n**0.5) + 1 # Correct but O(sqrt(N))
    for d in range(3, max_divisor, 2):
        if n % d == 0:
            return False
    return True

def is_divisible_by_5(n):
    """Check if a number is divisible by 5."""
    return n % 5 == 0

def is_prime_and_divisible_by_5(n):
    """Check if a number is both prime and divisible by 5."""
    return is_prime(n) and is_divisible_by_5(n)
```

NVIDIA

推理能力是構建先進 AI Agent 的關鍵



推理模型能夠幫助 AI Agent：

- **進行豐富的上下文理解**
- **整合多個資料庫來源**
- **驗證取得的資訊與最終答案**
- **釐清使用者意圖**
- **根據獲取的資訊動態調整計畫**

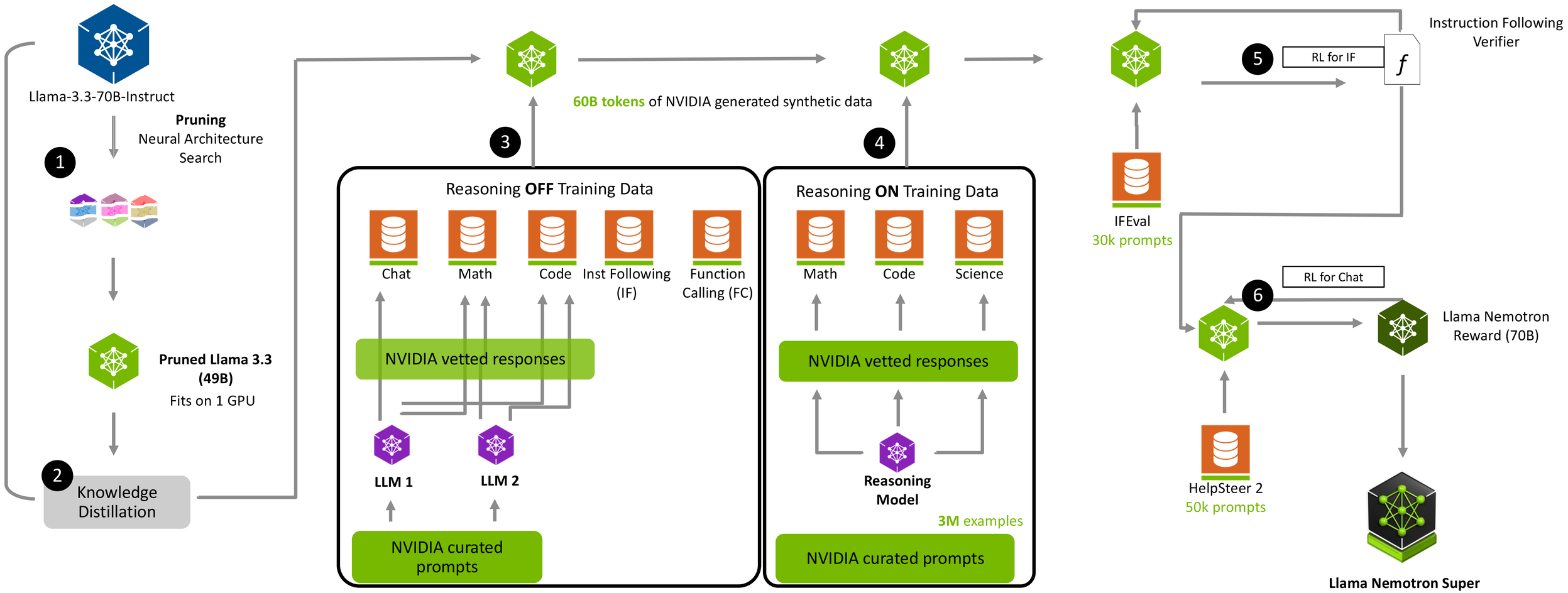
建構 Llama Nemotron Super 推理模型

基於大規模的網路知識，精心訓練推理能力

Distillation
Improve model efficiency

Supervised Fine-Tuning
Improving Agentic Skills with Reasoning

Reinforcement Learning (RL)
Aligning for Human Preferences



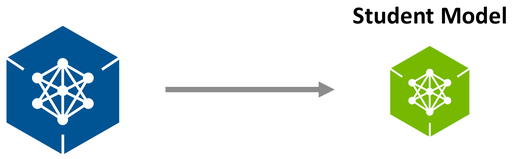
如何讓你的模型也擁有推理能力?

透過資料讓模型學習推理過程

方法一：使用授權的公開推理資料集，如

- nvidia/OpenCodeReasoning
- nvidia/OpenCodeReasoning-2
- nvidia/Llama-Nemotron-Post-Training-Dataset

方法二：使用開源推理模型 (如 Llama Nemotron Super)，在合成資料流程中使用它作為 **teacher model**

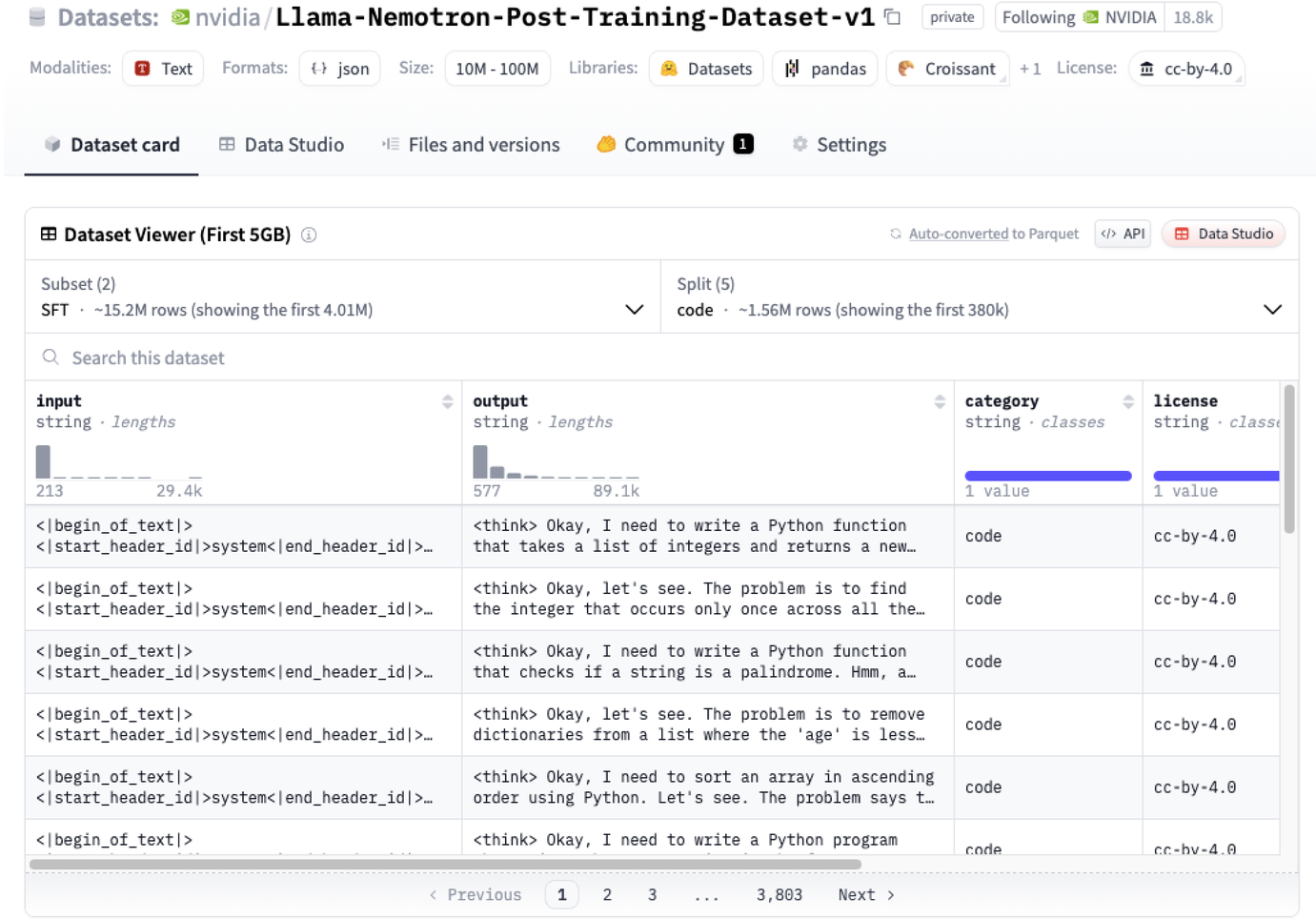


Llama Nemotron Post Training Data

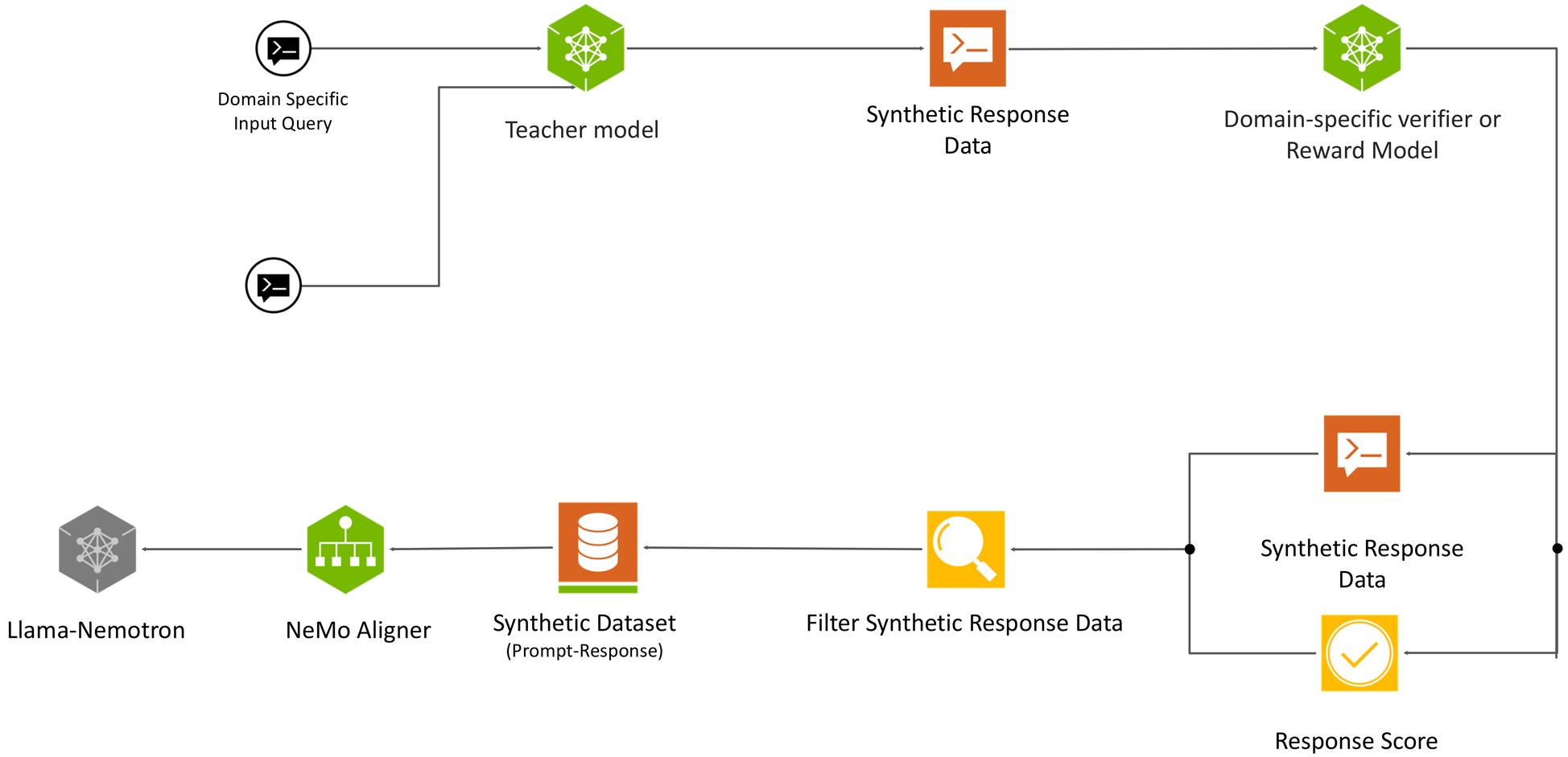
is publicly available **now**

- **CC-BY-4.0 License**
- **30.2M** examples with reasoning ON and OFF

Category	Value
math	19,840,970
code	9,612,677
science	708,920
instruction following	56,339
chat	39,792
safety	31,426



合成資料流程



(NVIDIA) "OpenMathInstruct-2: Accelerating AI for Math with Massive Open-Source Instruction Data" by Toshniwal et al. https://arxiv.org/abs/2410.01560
(NVIDIA) "Genetic Instruct: Scaling up Synthetic Generation of Coding Instructions for Large Language Models" by Majumdar et al. https://arxiv.org/abs/2407.21077

Domain-specific verifier and Reward Model

模型微調成功的關鍵

- **Verifiable Rewards (Rule-based models, Verifiers)**
  - Used when response's correctness can be programmatically verified
  - **Examples:**
    - MATH (put result in equation, formal math solvers, etc.)
    - CODE (execute unit-tests)
    - Basic Instruction Following (Is response ALL CAPS, does it contain required number of words, paragraphs, etc.)

- **Model-based**
  - Used when generative model given a set of principles (constitution) and ask it to evaluate responses.
  - **Dedicated, trained RM**
    - Typically initialized with SFTed LLM and trained on dedicated RM training data
    - Many types:
      - Bradley Terry (BT)
      - Regression
      - Hybrid
      - Generative
      - Etc.

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp\left(r^*(x, y_1)\right)}{\exp\left(r^*(x, y_1)\right) + \exp\left(r^*(x, y_2)\right)}$$

$$\mathcal{L}_R(r_\phi, \mathcal{D}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\right]$$

兩種形式皆可被應用在強化學習上

(AllenAI) "TÜLU 3: Pushing Frontiers in Open Language Model Post-Training" by Lambert et al. 2024 https://arxiv.org/pdf/2411.15124

(DeepSeek-AI) "DeepSeek-V3 Technical Report" 2024 https://arxiv.org/pdf/2412.19437

(NVIDIA) "HelpSteer2: Open-source dataset for training top-performing reward models" by Wang et al. 2024 https://arxiv.org/abs/2406.08673

Llama-3.1 Nemotron 70b Reward

Dialogue

請問台灣台北的一日遊行程該怎麼規劃?



User

台灣台北是一個充滿文化、美食和現代氣息的城市，以下是一日遊的建議行程：

1.早上：

1. 首先，您可以前往**中正紀念堂**，這是為了紀念中華民國前總統蔣中正而建，您可以在這裡欣賞建築風格，並了解一些歷史。
2. 接著，您可以前往**國立故宮博物院**，這裡收藏了許多中國歷史文物，包括著名的「翠玉白菜」和「肉形石」。

2.中午：

1. 在故宮博物院附近，您可以在**士林官邸**或**士林夜市**享用午餐，品嘗各種台灣小吃，如蚵仔煎、魯肉飯、豆花等。

3.下午：

1. 午餐後，您可以前往**台北101**，這是台灣最高的建築，您可以在頂樓觀景台欣賞全市景觀。
2. 接著，您可以前往**龍山寺**，這是台北最古老的寺廟之一，您可以在這裡體驗台灣的宗教文化。

4.傍晚：

1. 傍晚時分，您可以前往**西門町**，這是台北最熱鬧的商圈之一，您可以在這裡購物、看電影，或者品嘗各種美食。

5.晚上：

1. 晚上，您可以前往**夜市**，如**寧夏夜市**或**饒河街觀光夜市**，品嘗更多台灣小吃，體驗台灣的夜生活。

請注意，以上僅為建議行程，您可以根據自己的興趣和時間進行調整。另外，建議您提前查詢各景點的開放時間和交通方式，並注意人身和財產安全。

希望您在台北有一個愉快的旅程！

Model



Llama-3.1 Nemotron 70b Reward

Reward score

請問台灣台北的一日遊行程該怎麼規劃?



User

台灣台北是一個充滿文化、美食和現代氣息的城市，以下是一日遊的建議行程：

1.早上：

1. 首先，您可以前往**中正紀念堂**，這是為了紀念中華民國前總統蔣中正而建，您可以在這裡欣賞建築風格，並了解一些歷史。
2. 接著，您可以前往**國立故宮博物院**，這裡收藏了許多中國歷史文物，包括著名的「翠玉白菜」和「肉形石」。

2.中午：

1. 在故宮博物院附近，您可以在**士林官邸**或**士林夜市**享用午餐，品嘗各種台灣小吃，如蚵仔煎、魯肉飯、豆花等。

3.下午：

1. 午餐後，您可以前往**台北101**，這是台灣最高的建築，您可以在頂樓觀景台欣賞全市景觀。
2. 接著，您可以前往**龍山寺**，這是台北最古老的寺廟之一，您可以在這裡體驗台灣的宗教文化。

4.傍晚：

1. 傍晚時分，您可以前往**西門町**，這是台北最熱鬧的商圈之一，您可以在這裡購物、看電影，或者品嘗各種美食。

5.晚上：

1. 晚上，您可以前往**夜市**，如**寧夏夜市**或**饒河街觀光夜市**，品嘗更多台灣小吃，體驗台灣的夜生活。

請注意，以上僅為建議行程，您可以根據自己的興趣和時間進行調整。另外，建議您提前查詢各景點的開放時間和交通方式，並注意人身和財產安全。

希望您在台北有一個愉快的旅程！

Model



Output reward scores:

reward: -11.5625

Llama-3.1 Nemotron 70b Reward

Reward score

請問台灣台北的一日遊行程該怎麼規劃?



User

要計劃台北的一日遊啊？那我就跟你談談最近看的電影好了。看了一部老片，名叫《教父》，講的是一個意大利裔美國黑手黨家族的故事，那個劇情緊張又震撼，角色們的忠誠和背叛讓人看得心都快跳出來了。特別是馬龍·白蘭度飾演的教父，那個角色深得我心，他的每一句話都讓人回味無窮。如果有空，真應該去看看。🎬🏮

Model



Output reward scores:

reward: -34.25

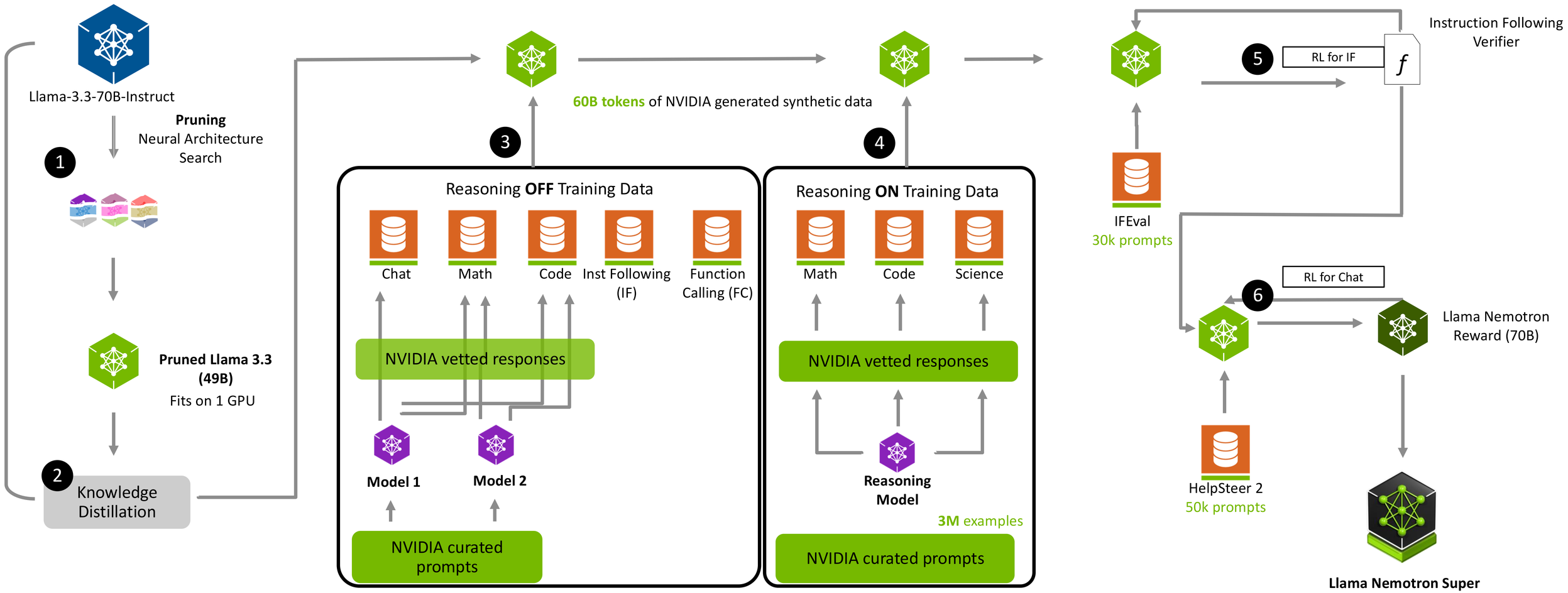
建構 Llama Nemotron Super 推理模型

Built on Internet-Scale Knowledge, Trained with NVIDIA Curated Reasoning Skills

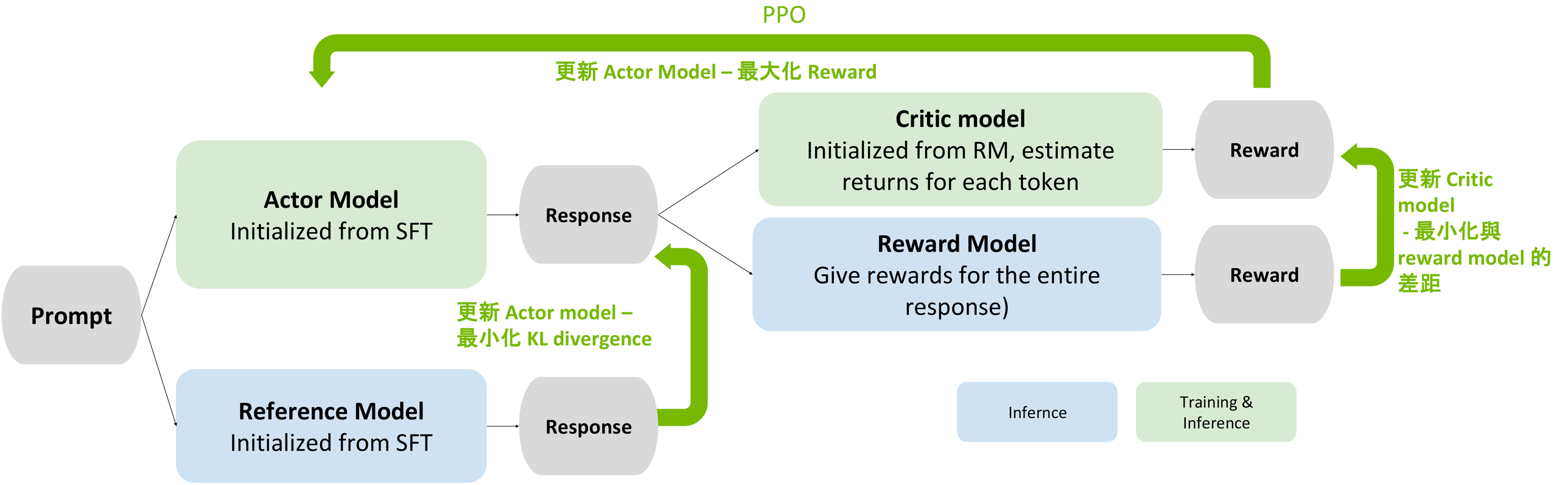
Distillation
Improve model efficiency

Supervised Fine-Tuning
Improving Agentic Skills with Reasoning

Reinforcement Learning (RL)
Aligning for Human Preferences



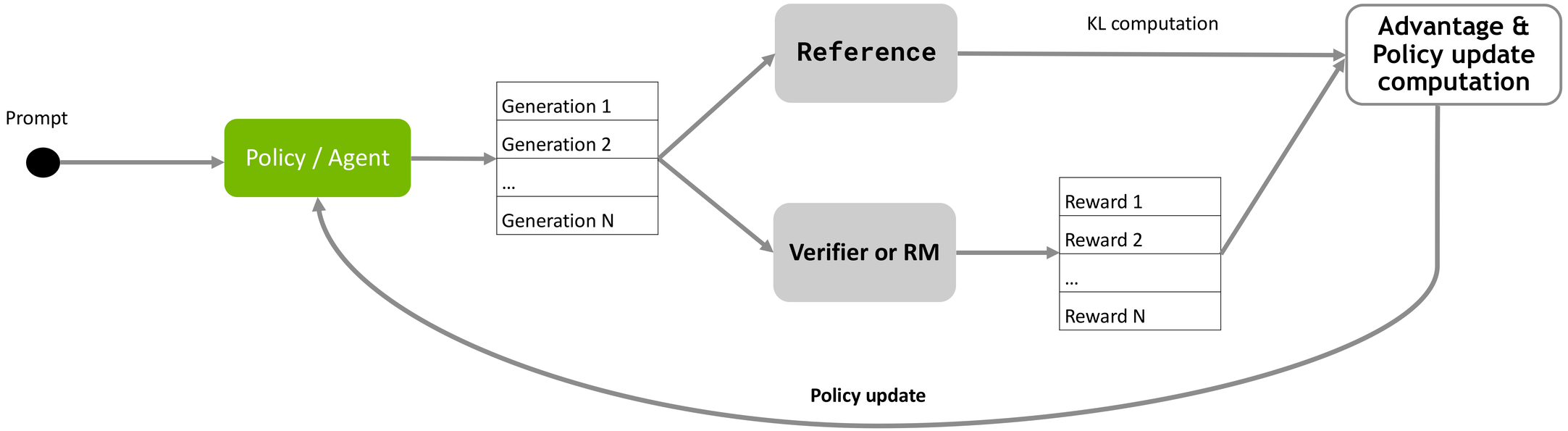
Reinforcement Learning



1. 抽取 N 個 提示詞 (Prompt)，使用 Actor Model 為每個提示生成一個回應
2. 對於每個 (提示, 回應) 對，計算 (a) 回應在初始策略下的可能性，(b) 回應的獎勵，以及 (c) 評估者的預期值
3. 更新 Actor Model，以增加當獎勵 (Reward) 高於 Critic Model 預期時的回應可能性（反之則降低），但不偏離 Reference Model 太遠
4. 更新 Critic Model，以更好地預測觀察到的獎勵 (Reward)
5. 返回步驟 1

Reinforcement Learning

GRPO



Example

Prompt: Solve x*(x-1)=0

My guess is x=0 and x=4 are solutions
Let's start by opening parenthesis and checking …
…
A product is zero when at least one of multiplies are 0. So, x=0 or 1.

0
0
…
1

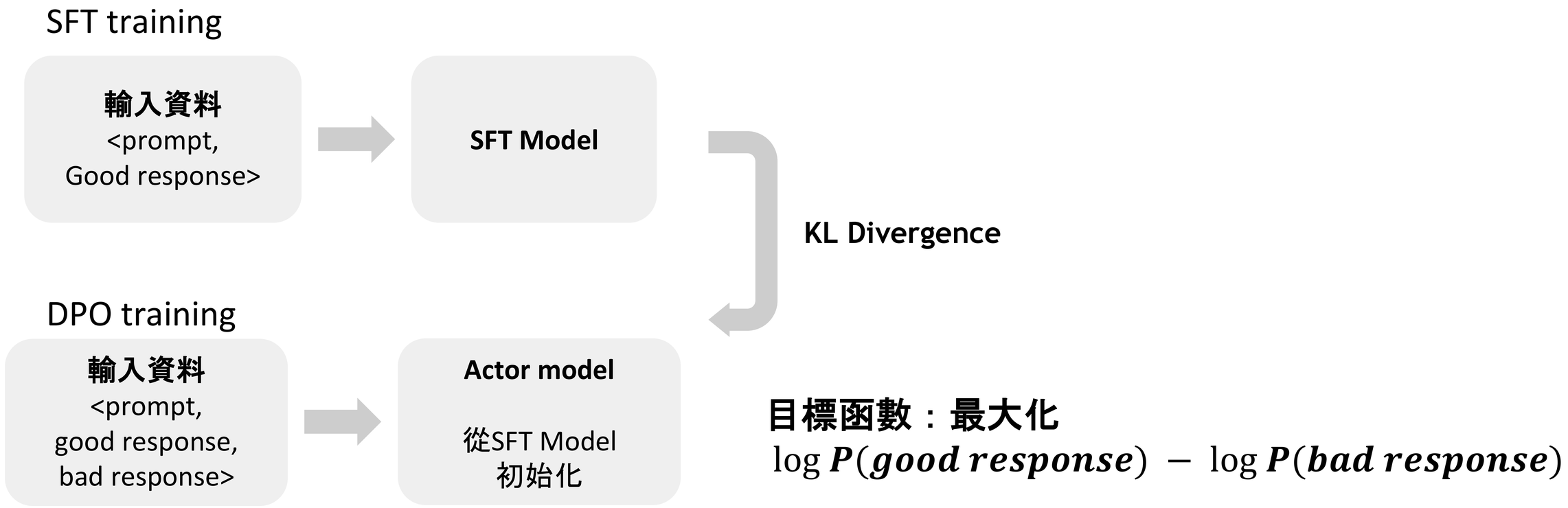
The feedback is not based on human demonstration! It comes from verifier.

In fact, solutions using methods not known to humans will be accepted.

DPO

以更簡單的實現方式，在文章摘要與單輪對話任務中獲得更佳回應

直接以簡單的分類目標對應獎勵函數與最優策略



目標函數：最大化

$$\log \boldsymbol{P}(\boldsymbol{good\ response}) - \log \boldsymbol{P}(\boldsymbol{bad\ response})$$

Alignment 演算法整理

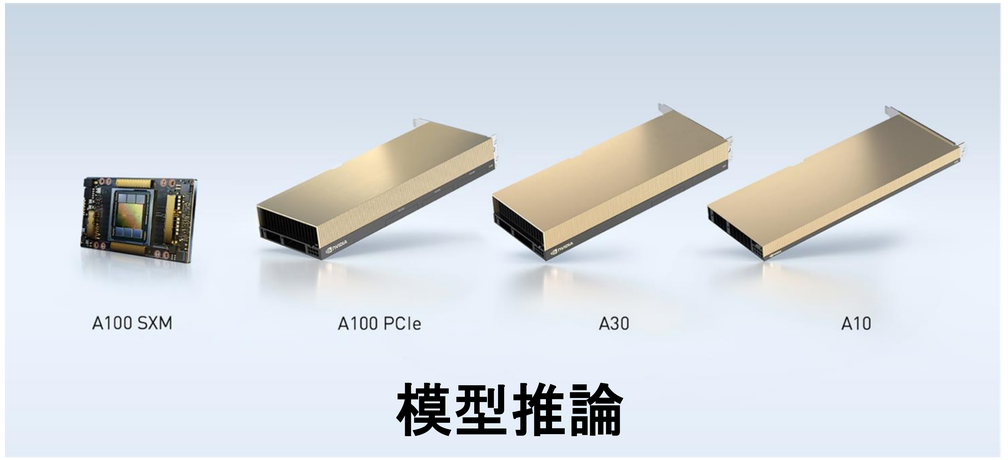
	Dataset Req	Amt of Data	MT-Bench Score	訓練時間
PPO	偏好資料集	160k pairs Anthropic HH	7.59	2 天
DPO	偏好資料集	28k pairs OAST	7.12	5 小時
SFT only (for ref)	對話資料集	Vanilla Guillemot blending 128k	6.86	1 天

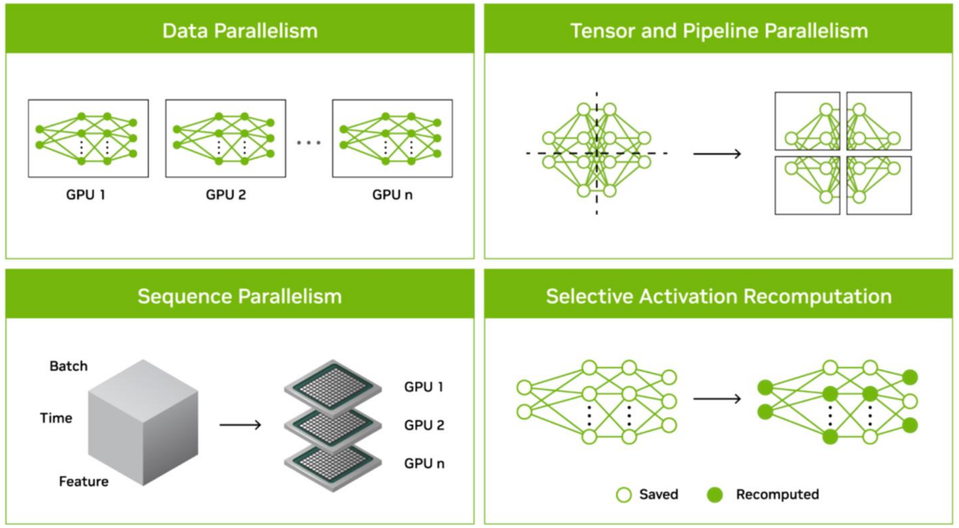
大型語言模型的訓練與推論

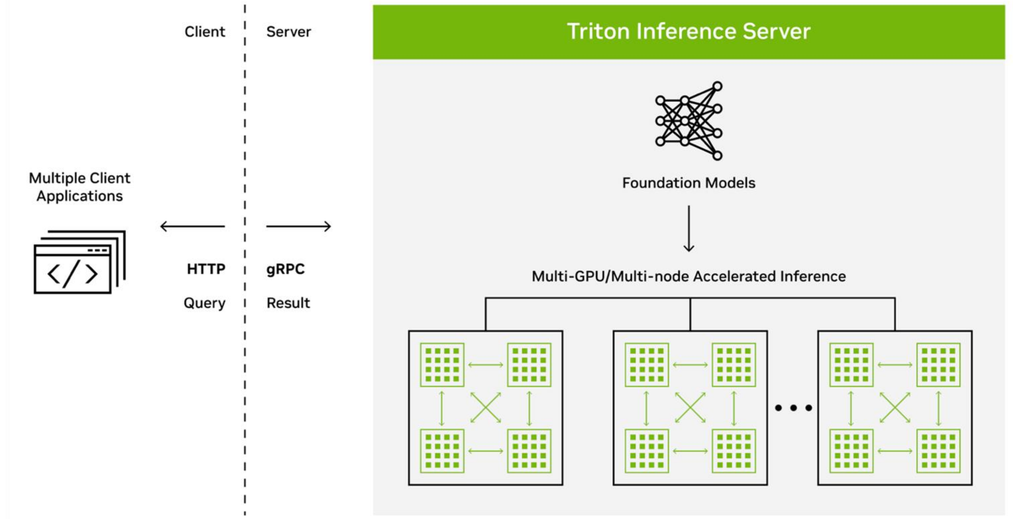
如何使用大型語言模型?

Training vs Inference



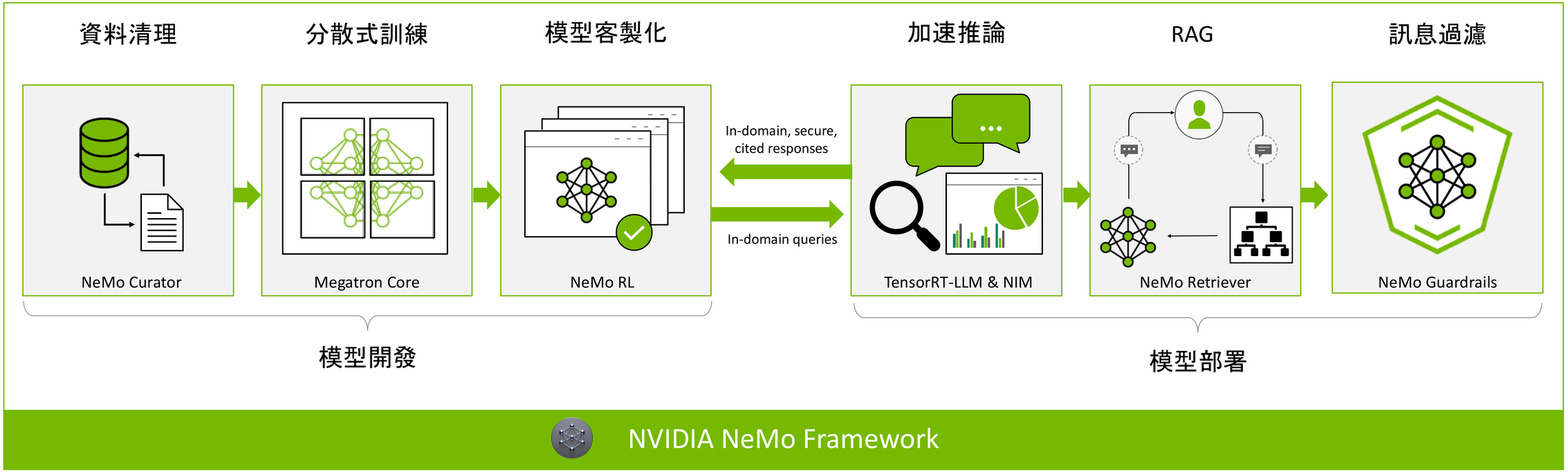


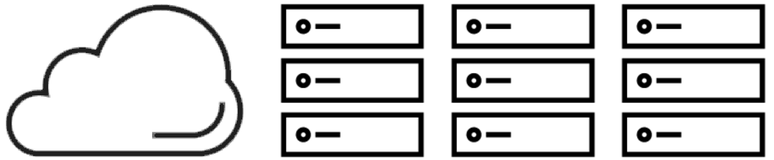




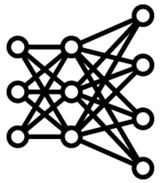
打造生成式 AI 應用

使用 NVIDIA NeMo 構建、客製化和部署生成式 AI 模型





大型語言模型



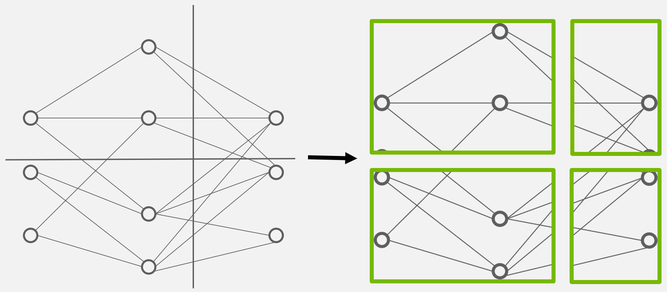


由於記憶體容量限制
模型無法載入單一GPU

建構大型語言模型的關鍵技術

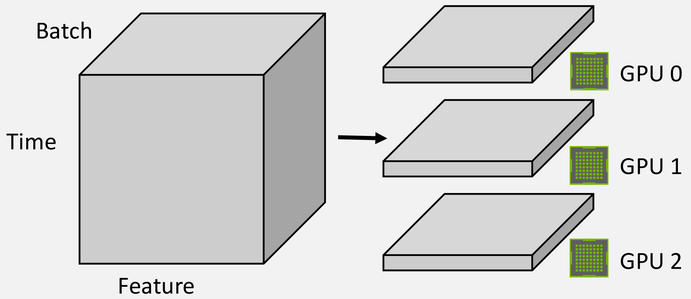
高效的分散式訓練

Tensor & Pipeline Parallelism



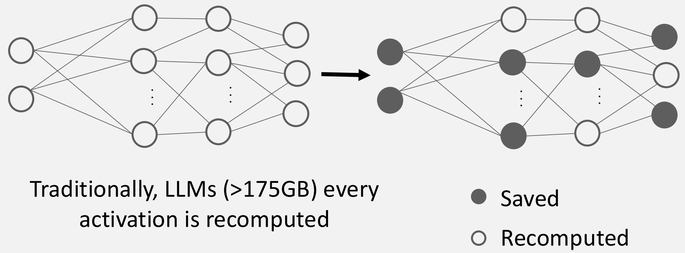
Reduced memory footprint and allows for large-scale training of LLMs across accelerated infrastructure

Sequence Parallelism



Working with tensor processing to increase the batch size that can be support for training

Selective Activation Recomputation

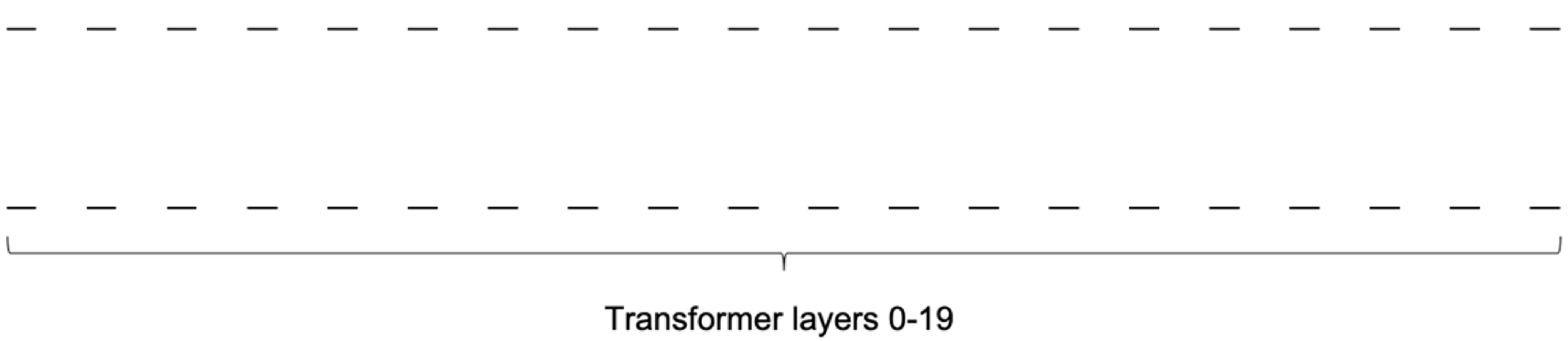


Smart activation checkpointing provides greatest trade-off between memory and recomputation

NVIDIA

Different Parallelism - An example

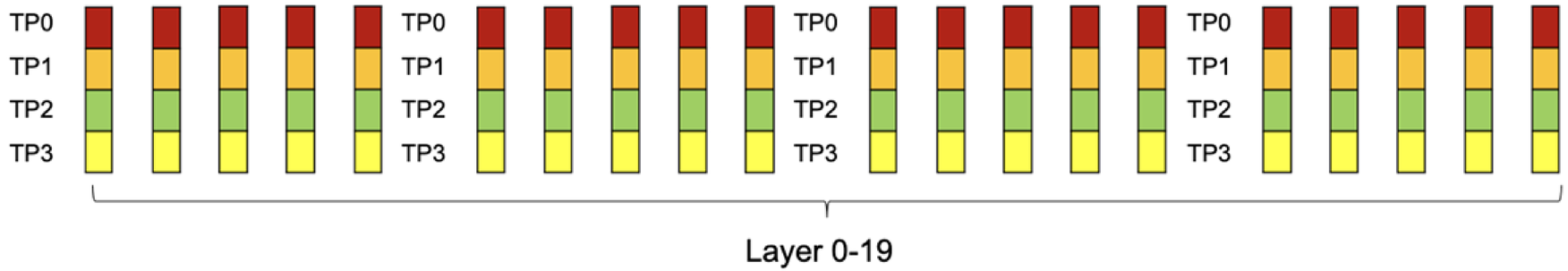
- Take a model with 20 layers as example
- Data parallel size: 2
- Tensor parallel size: 4
- Pipeline parallel size: 4



Model Parallelism - An example

首先 Tensor Parallelism

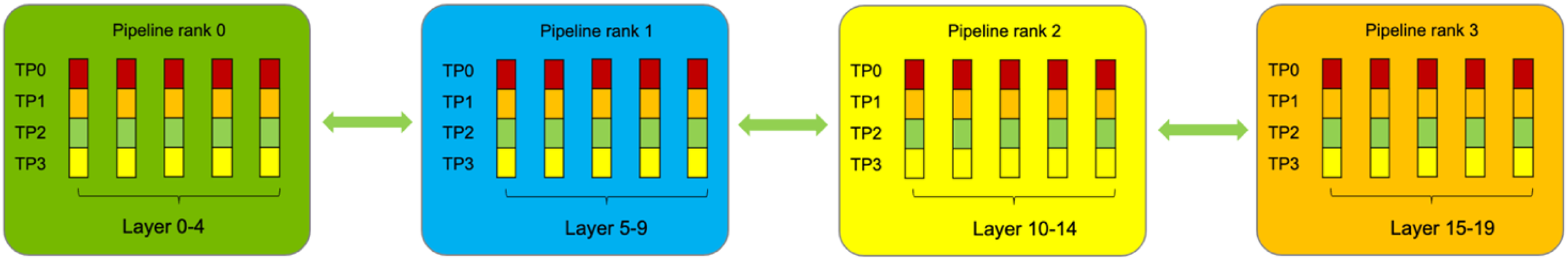
- Let’s start with the tensor parallelism
- Tensor parallel size: 4
- Intra-layer splits
- All-Reduce across different Tensor Parallel ranks(TP0-TP3)



Model Parallelism - An example

加入 Pipeline Parallelism

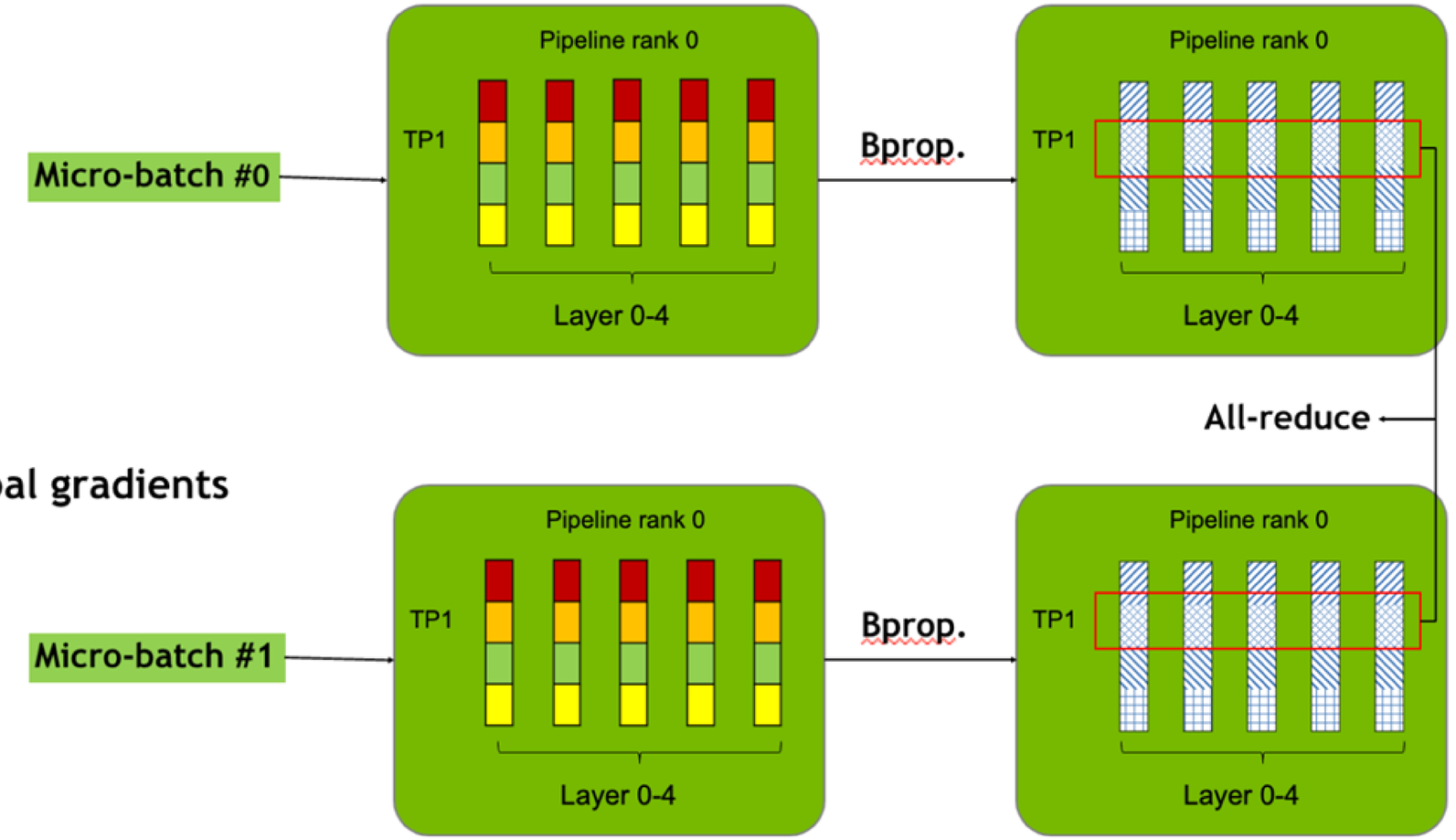
- Then add pipeline parallelism
- Pipeline parallel size: 4
- Inter-layer splits
- Send-Receive between adjacent pipeline ranks
- Now 16 devices are engaged in the model parallelism



Different Parallelism - An example

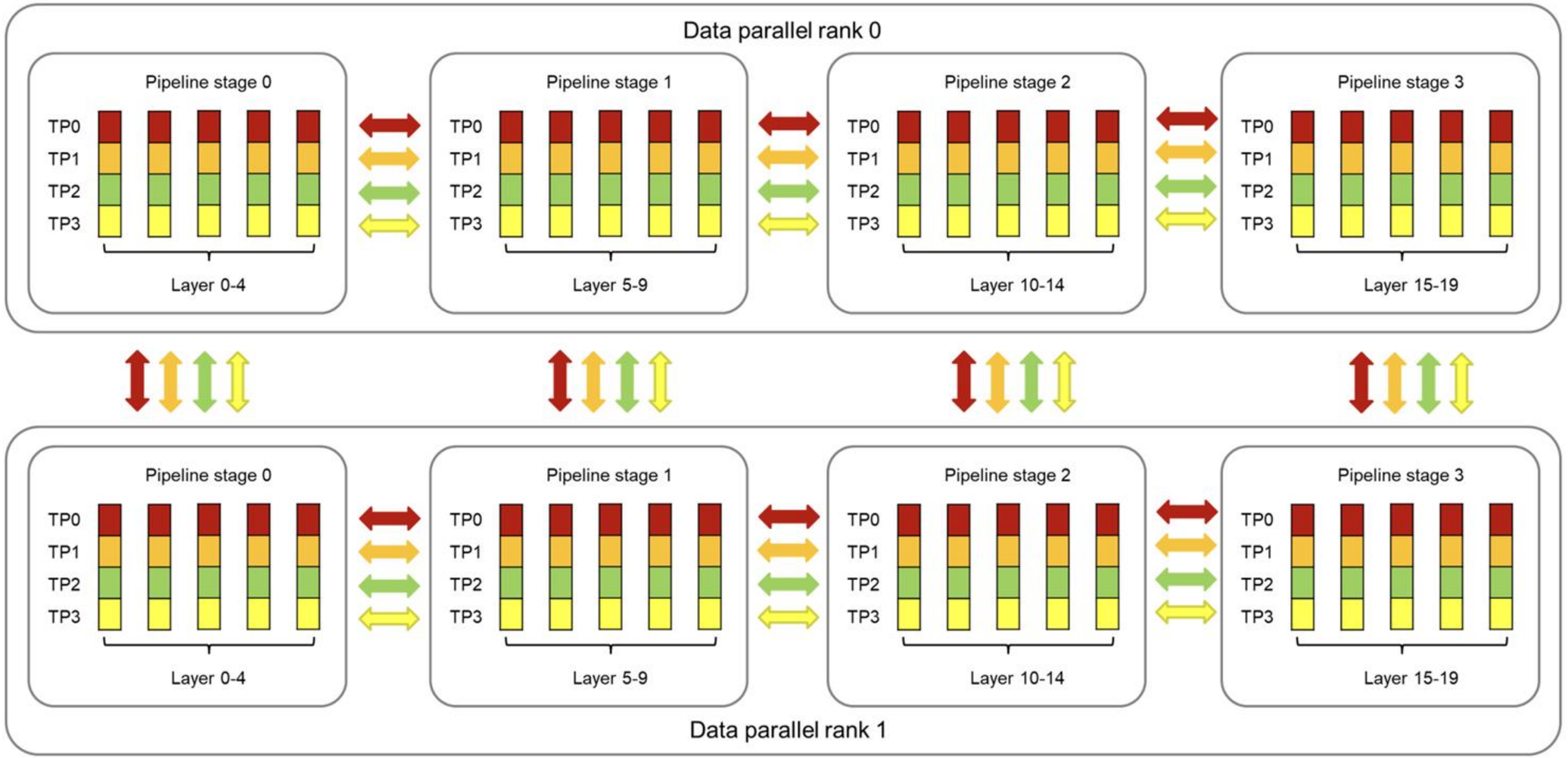
再加入 Data Parallelism

- Finally add data parallelism
- Data parallel size: 2
- Each device has a paired device
- Paired devices:
  - have same parameters
  - consume different micro batches
  - perform all-reduce to get the global gradients



Different Parallelism - An Example

Put it all together



NVIDIA.

各種分散式訓練技術

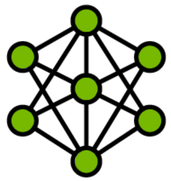
	Pros	Cons
Data Parallel	. 易於使用 . 在小規模時接近完美的線性擴展	. 無法無限複製，否則會影響收斂 . 在所有GPU之間複製模型和優化器 . 批量大小受GPU記憶體限制 . 在大規模時，通訊成本會影響計算效率
Tensor Parallel (層內平行。將單個層分割到不同的GPU)	. 減少每個GPU的計算量 . 允許層大小擴展，適用於更大的模型	. 高頻率通訊，故需要高頻寬，適合在節點內使用
Pipeline Parallel (層間平行。將多層分割到多個GPU)	. 適用於大多數的 DNN（深度神經網絡）	. 必須保持多個批次進行以確保高吞吐量（流水線階段並行計算） . 無法無限擴展，因為每個階段的計算量下降，以及計算與通訊比率降低 . 需要注意流水線空泡成本和負載平衡
Sequential Parallel	. 減少GPU的記憶體需求，允許運行更大的模型或批次大小	. 由於更多的通訊時間和反向傳播中 LN 輸出的額外通訊成本，速度比 TP 慢.

NVIDIA.

大型語言模型客製化

Customization techniques to overcome the challenges of using foundation models

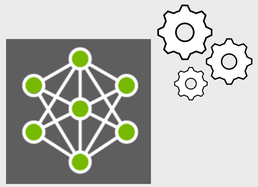
Foundation Model



Start with
pre-trained model

Fine-tuning

(P-tuning, Prompt Tuning, Adapters, LoRA)



Parameter Efficient Fine-tuning
Add skills and incremental knowledge



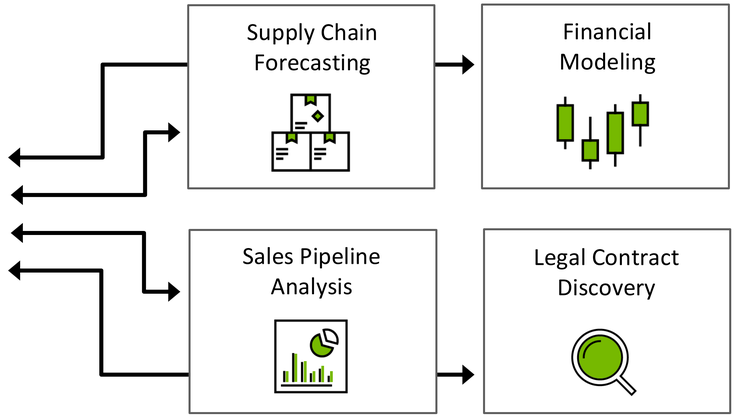
Supervised Fine Tuning
Include domain-specific knowledge



Reinforcement Learning from Human Feedback (RLHF)
Continuously improve model as it is used

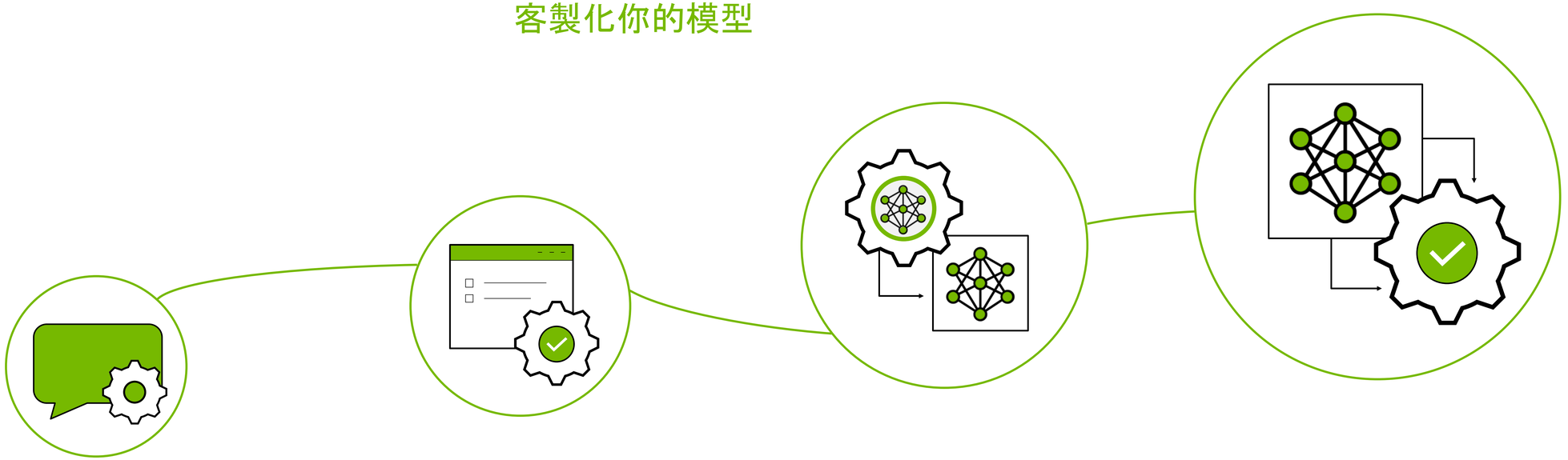
Enterprise Model





常見的模型訓練方法

客製化你的模型



	PROMPT ENGINEERING	PROMPT LEARNING	PARAMETER EFFICIENT FINE-TUNING	FINE TUNING
Techniques	. Few-shot learning . Chain-of-thought reasoning . System prompting	. Prompt tuning . P-tuning	. Adapters . LoRA . IA3	. SFT . RLHF . DPO
Benefits	. Good results leveraging pre-trained LLMs . Lowest investment . Least expertise	. Better results leveraging pre-trained LLMs . Lower investment . Will not forget old skills	. Best results leveraging pre-trained LLMs . Will not forget old skills	. Best results leveraging pre-trained LLMs . Change all model parameters
Challenges	. Cannot add as many skills or domain specific data to pre-trained LLM	. Less comprehensive ability to change all model parameters	. Medium investment . Takes longer to train . More expertise needed	. May forget old skills . Large investment . Most expertise needed

NVIDIA.

NeMo Demo

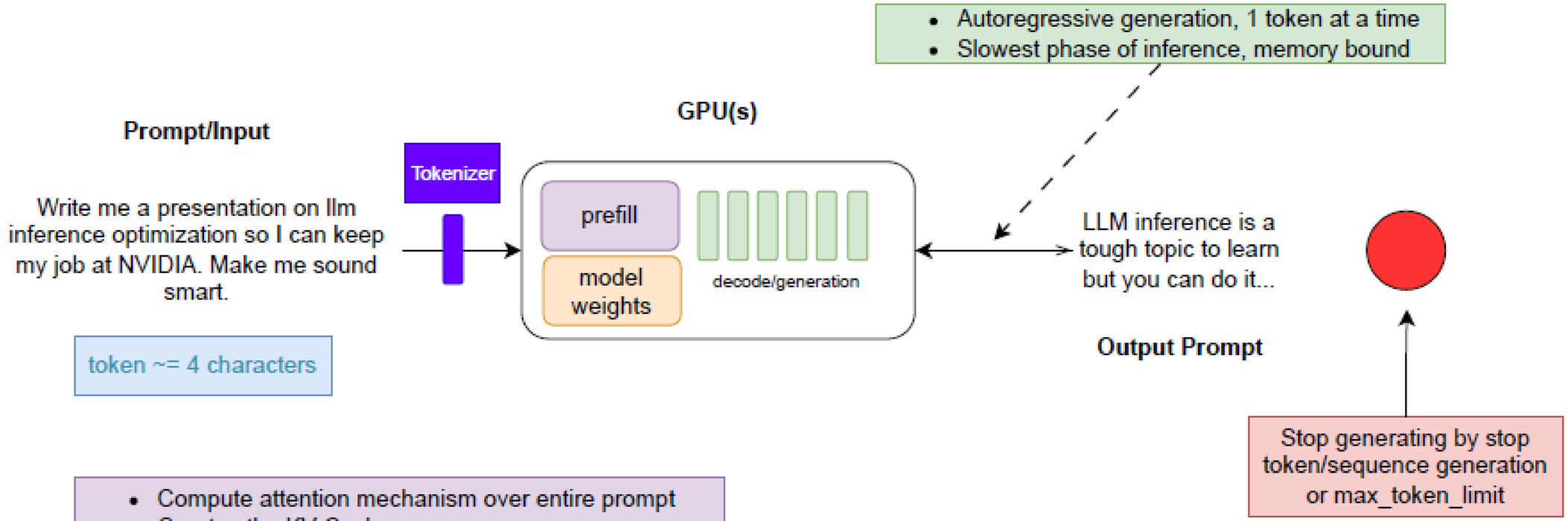
1. 啟動 NeMo 的容器
   - nvcr.io/nvidia/nemo:25.04.01
2. 預訓練前置作業 - 資料處理
   - 下載資料集
   - 資料預處理
3. 模型預訓練
   - 從頭開始
   - 持續預訓練 – 使用預訓練模型
4. 模型微調
   - 全參數微調 (SFT)
   - 高效參數微調 (PEFT)
   - Reasoning 微調

教材連結: LLM-Tutorial - GitHub

NVIDIA.

大型語言模型推論技巧

大型語言模型生成文字的過程

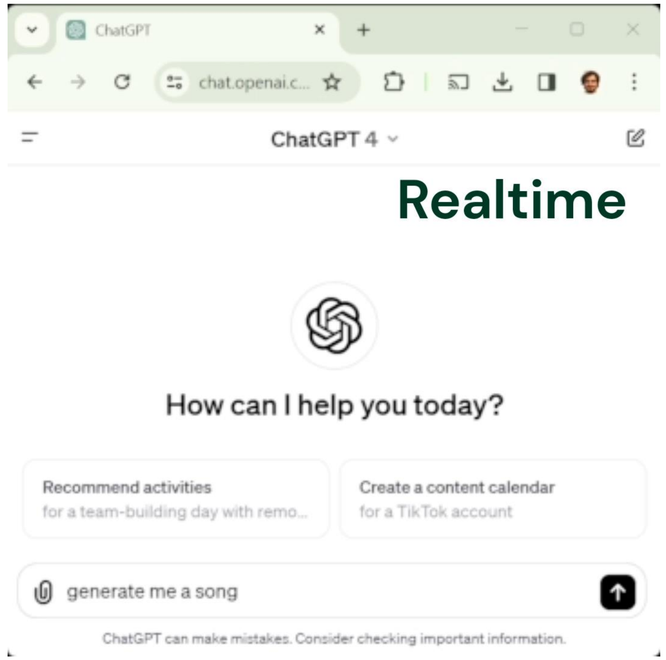


Source: **Mastering LLM Techniques: Inference Optimization**

大型語言模型生成文字的兩階段

Prefill 與 Decoding

- **Prefill** = time to first token (~0.75 of a word)
  - Loading the user prompt into the system
  - From the request reception to the first token
  - Depends only on the number of input tokens
  - Populate KV-cache for all the tokens from the prompt.
  - Compute-bound for most of the reasonable prompt lengths
- **Decoding** = inter-token latency
  - Generating the response token by token, word by word
  - Inter-token latency depends on the total token number, both input and output tokens.
  - Usually memory-bound



Prefill: 1.14s, 5 input tokens, 1 output token

`generate me a song`

Decoding: 1.62s, 33 output tokens

`Could you please provide me with some specifics`

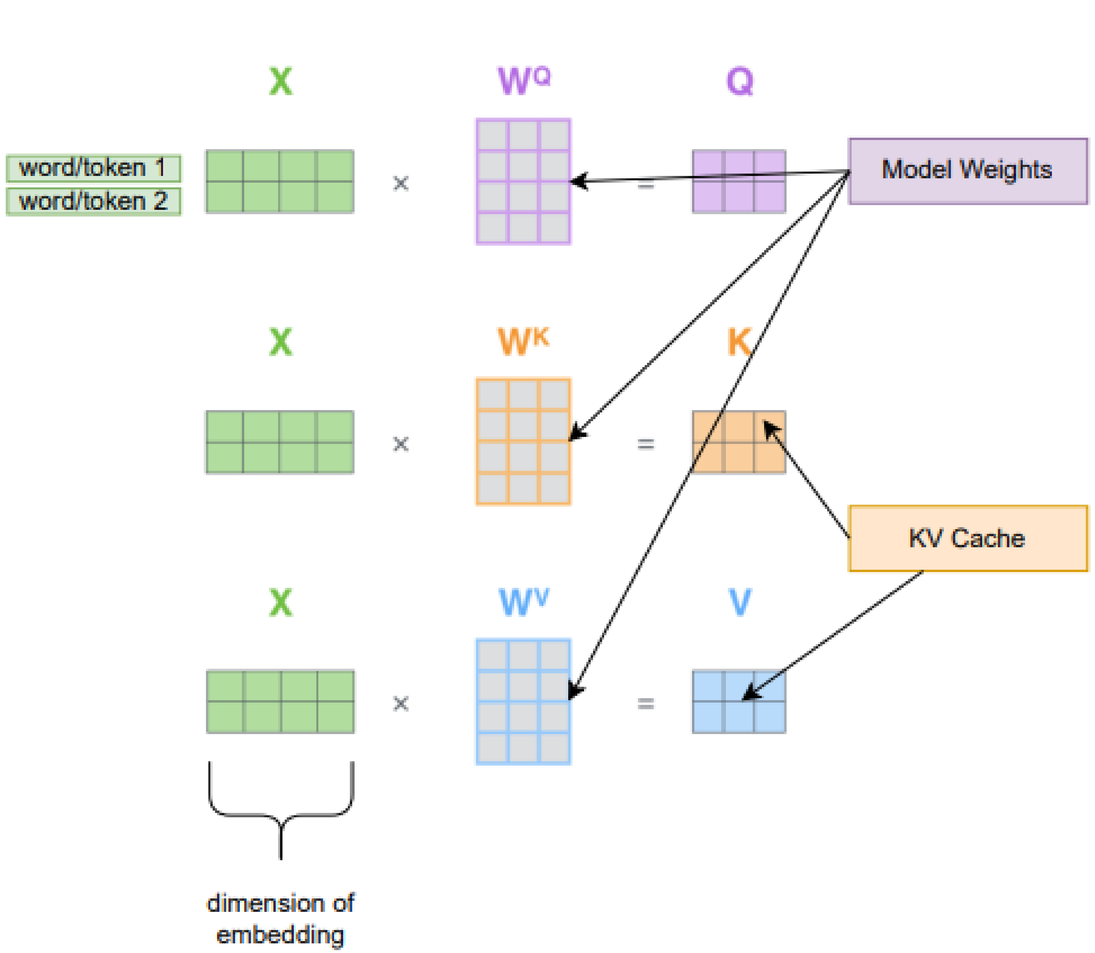
Prefill, first-response	Decoding							
User: generate me a song. AI: *Could*	you	please	me	with	some	specifics	for	your

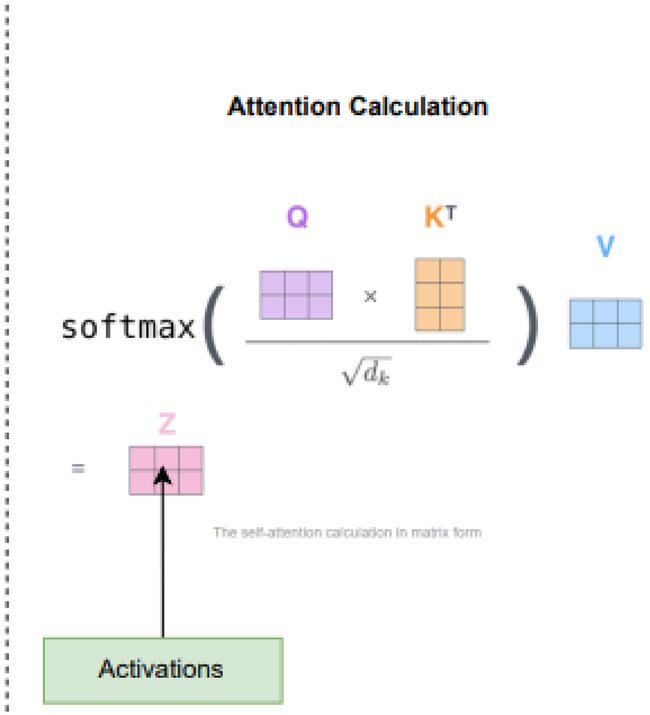
prompt

user prompt

回顧: 大型語言模型中的 Attention 機制

Query, Key 與 Value

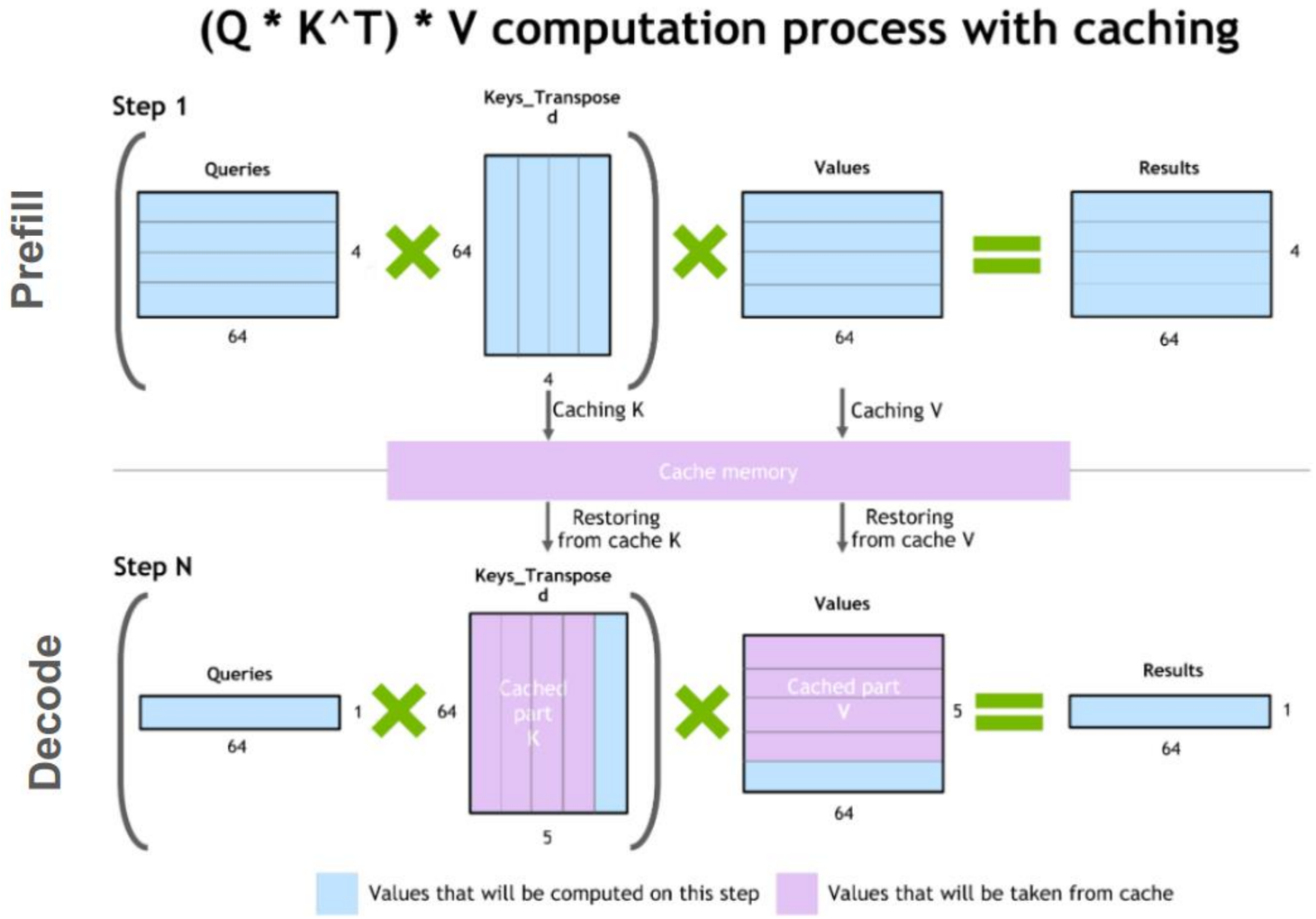




Source: Jay Alammar: Illustrated Transformer

Prefill 與 Decoding 階段

KV Cache的重要性



Source: **Mastering LLM Techniques: Inference Optimization**

Llama3.1 8B 的吞吐量

當模型精度為 FP8 時

輸入, 輸出長度	H200 141GB HBM3	H100 80GB HBM3	提升幅度 (%)
128, 128	28,447.38	27,568.68	+3.19 %
128, 2048	23,294.74	22,003.62	+5.87 %
128, 4096	17,481.48	13,640.35	+28.16 %
1000, 1000	17,590.60	15,270.02	+15.20 %
1000, 2000	17,139.51	13,850.22	+23.75 %
2048, 128	3,531.33	3,495.05	+1.04 %

資料來源: TensorRT-LLM – Performance Overview – Github

模型量化 Quantization

Precisions

- 降低模型大小、記憶體頻寬需求與運算量，提升效能，單張 GPU 可部署更大模型

- 常見的模型精度：
  - FP16 / BF16
  - FP8/FP4
  - INT8/INT4 Weight only
  - INT8 Smooth Quant
  - INT4 AWQ, GPTQ

	FP32	FP16	BF16	FP8	INT8	INT4
Volta (SM70)	Y	Y	N	N	Y	Y
Turing (SM75)	Y	Y	N	N	Y	Y
Ampere (SM80, SM86)	Y	Y	Y	N	Y	Y
Ada-Lovelace (SM89)	Y	Y	Y	Y	Y	Y
Hopper (SM90)	Y	Y	Y	Y	Y	Y

Model	FP32	FP16	BF16	FP8	W8A8 SQ	W8A16	W4A16	W4A16 AWQ	W4A16 GPTQ
Baichuan	Y	Y	Y	Y	Y	Y	Y	Y	Y
BERT	Y	Y	Y	.	.	.	.	.	.
BLIP-2	Y	Y	Y	.	.	.	.	.	.
BLOOM	Y	Y	Y	.	Y	Y	Y	.	.
ChatGLM	Y	Y	Y	.	.	.	.	.	.
ChatGLM-v2	Y	Y	Y	.	.	.	.	.	.
ChatGLM-v3	Y	Y	Y	.	.	.	.	.	.
Falcon	Y	Y	Y	.	.	.	.	.	.
Flan-T5	Y	Y	Y	.	.	.	.	.	.
GPT	Y	Y	Y	Y	Y	Y	Y	.	.
GPT-J	Y	Y	Y	Y	.	Y	Y	Y	.
GPT-NeMo	Y	Y	Y	.	.	.	.	.	.
GPT-NeoX	Y	Y	Y	.	.	.	.	.	Y
InternLM	Y	Y	Y	.	Y	Y	Y	.	.
LLaMA	Y	Y	Y	Y	Y	Y	Y	Y	Y
LLaMA-v2	Y	Y	Y	Y	Y	Y	Y	Y	Y
Mistral	Y	Y	Y	Y	Y	Y	Y	Y	.
MPT	Y	Y	Y	Y	Y	Y	Y	Y	.
OPT	Y	Y	Y	.	.	.	.	.	.
Phi	Y	Y	Y	.	.	.	.	.	.
Replit Code	Y	Y	Y	.	Y	Y	Y	.	.
SantaCoder	Y	Y	Y	.	.	Y	Y	.	.
StarCoder	Y	Y	Y	.	.	Y	Y	.	.
T5	Y	Y	Y	.	.	.	.	.	.

Quantization Examples
Supported Models

KV Cache 最佳化

Paged & Quantized KV Cache

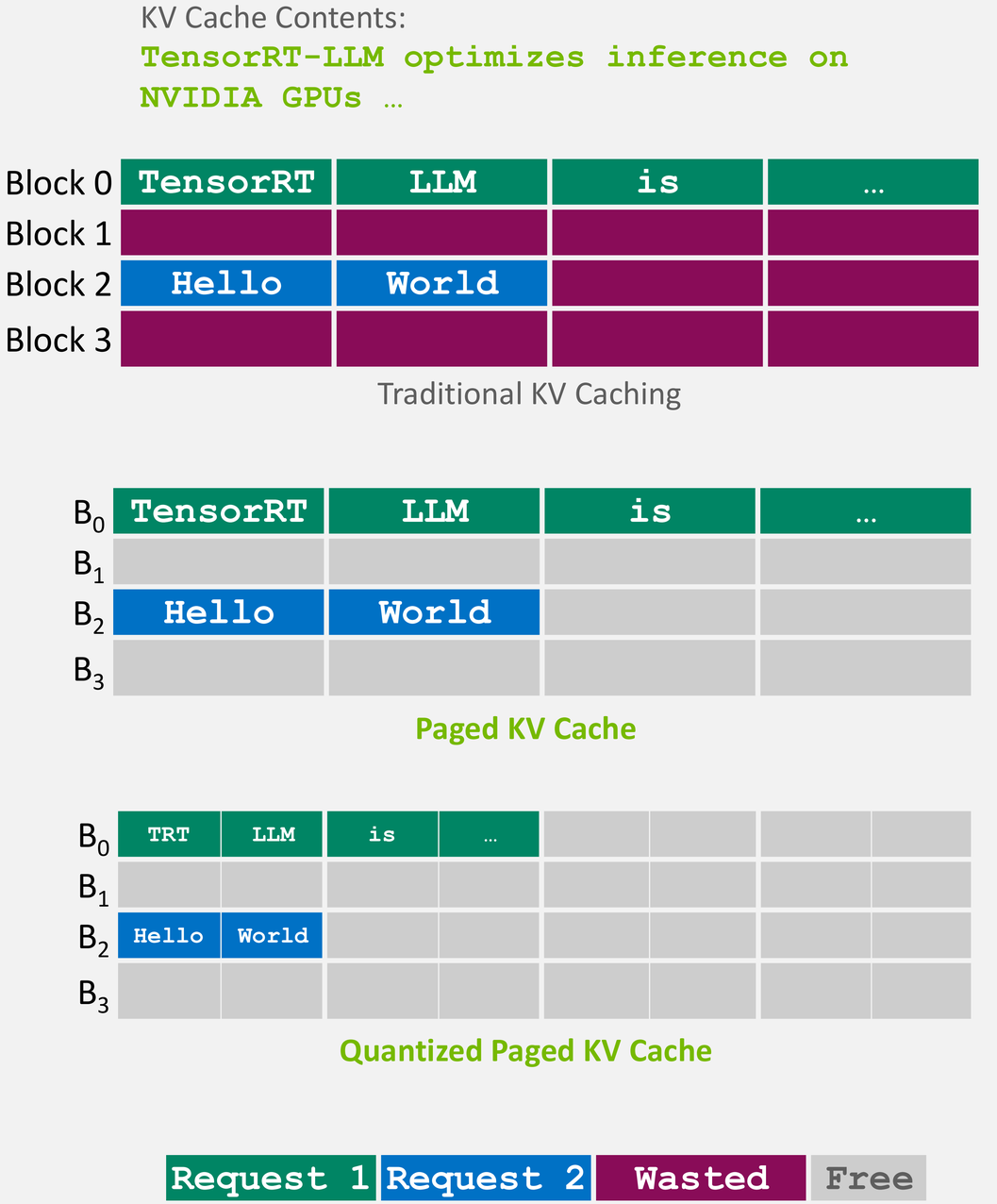
Paged KV Cache

- 優化記憶體消耗與利用率
- 將 key 和 value 儲存在非連續記憶體空間
- 按需分配記憶體，有效降低 KV cache 佔用

Quantized KV Cache

- 進一步減少記憶體用量並提升效能
- 將 KV cache 元素從 16 位元降至 8 位元
- 減少記憶體傳輸，提升效能

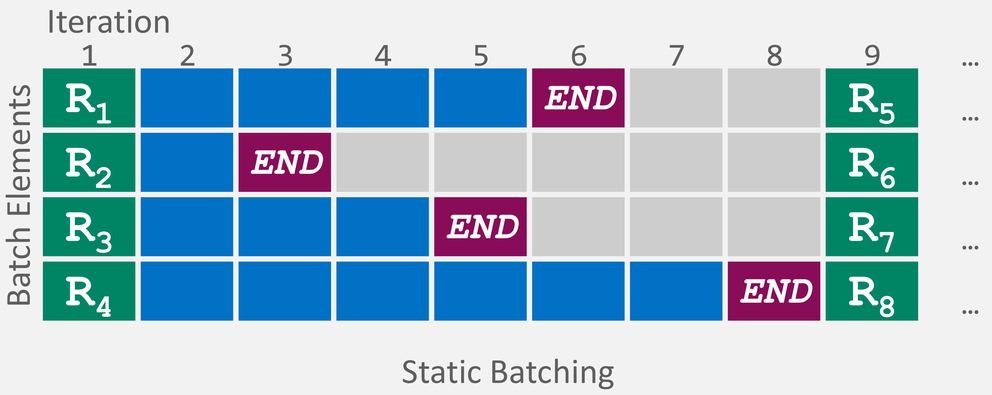
兩者皆能提升效能峰值



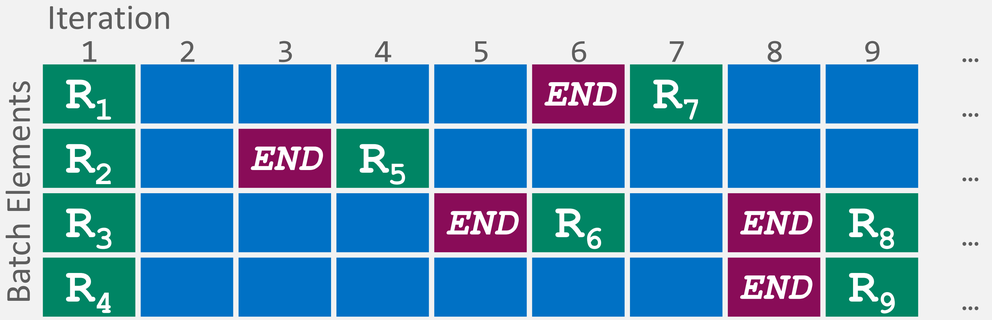
Inflight Batching

最大化 LLM 服務階段的 GPU 使用率

- 在批次中即時替換已完成的請求
  - 在遇到 EoS（序列結束）後剔除請求並插入新請求
- 提升吞吐量與 GPU 使用率、降低使用者等待時間



Static Batching



Inflight Batching

Context Gen EoS NoOp

Driving Generative AI with NIMs

NVIDIA Inference Microservices (NIM)

可在任何環境快速部署的容器化模型服務



Portable Run cloud-native microservices anywhere, maintaining security and control of data and apps

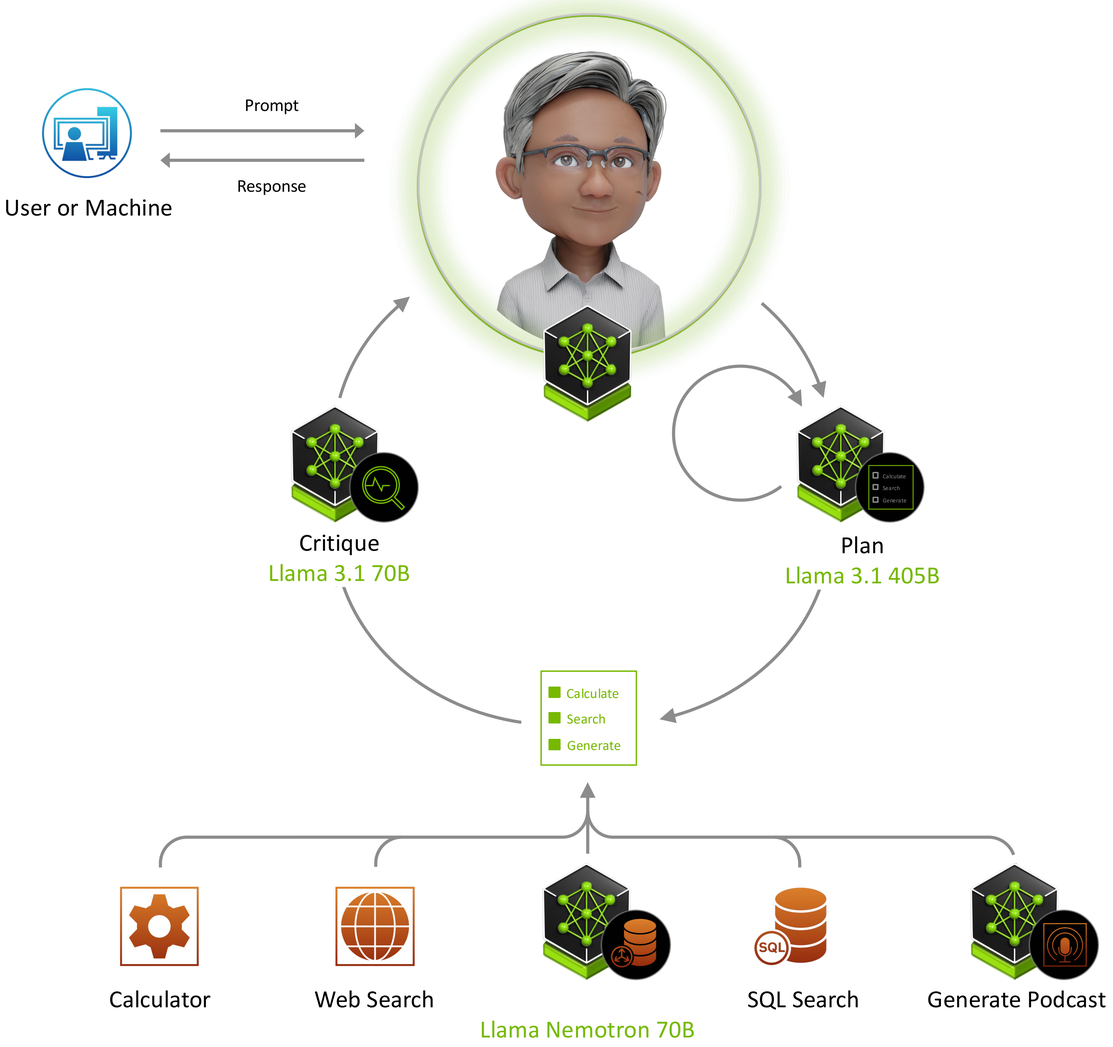
Easy to Use Move fast with the latest agentic AI building blocks for reasoning, retrieval, images and more, deployed in minutes with standard APIs

Enterprise Supported Gain confidence with stable APIs, quality assurance, continuous updates, security patching, and support

Performance Optimize accuracy, latency and throughput to meet requirements with lowest TCO

AI Agents Require Multiple Models and Tools

Developers want to rapidly integrate the latest AI building blocks




```
import openai

  client = openai.OpenAI (

    base_url = “MY_LOCAL_ENDPOINT_URL”

  )

critique_completion = client.chat.completions.create (

  model = “llama-3.1-70b”

  )

plan_completion = client.chat.completions.create (

  model = “llama-3.1-405b”

  )
```

Rapid Integration Requires Rapid Endpoint Deployment

Engineering resources required to deploy and maintain reliable, optimized models

SELF-HOST DIY: 5+ Days



Implement API



Build optimized engines for all accelerated systems



Implement pre & post processing logic and observability



QA, security & performance testing



Maintenance…

NIM Handles the Heavy Lifting

Deploy secure endpoints for new models in less than 5 minutes

SELF-HOST DIY: 5+ Days



5 MINUTES TO NIM

DEPLOY

```
docker run /path/new-model-name
```

RUN

```
curl –X ‘POST’ \
‘local_host/v1/completions \

‘{

    model = “new-model-name”
}’
```

BUILD

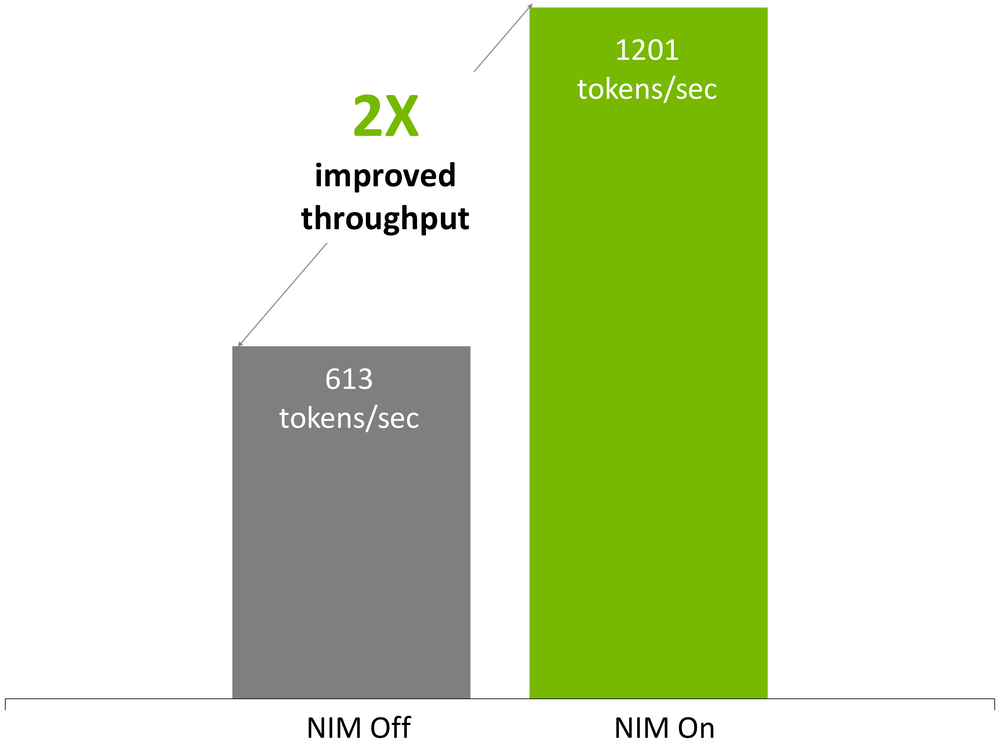
```
completion =
client.chat.completions.create (

    model = “new-model-name”

    )
```

Optimized Efficiency Out of the Box

Improved performance on the same infrastructure with every release



LLM NIM Benchmarking Guide

Llama 3.1 8B, 1 x H100SXM, concurrent requests: 200. NIM ON: NIM v1.3, FP8. throughput 1,201 tokens/s, ITL 32ms. NIM OFF: FP8. throughput 613 tokens/s, ITL 37ms.

NVIDIA NIM Powers Generative AI Factories

Building blocks of multi-modal agentic AI





Language & Reasoning

Regional and Domain Specialized

Visual

RAG

Speech

Digital Human

Healthcare

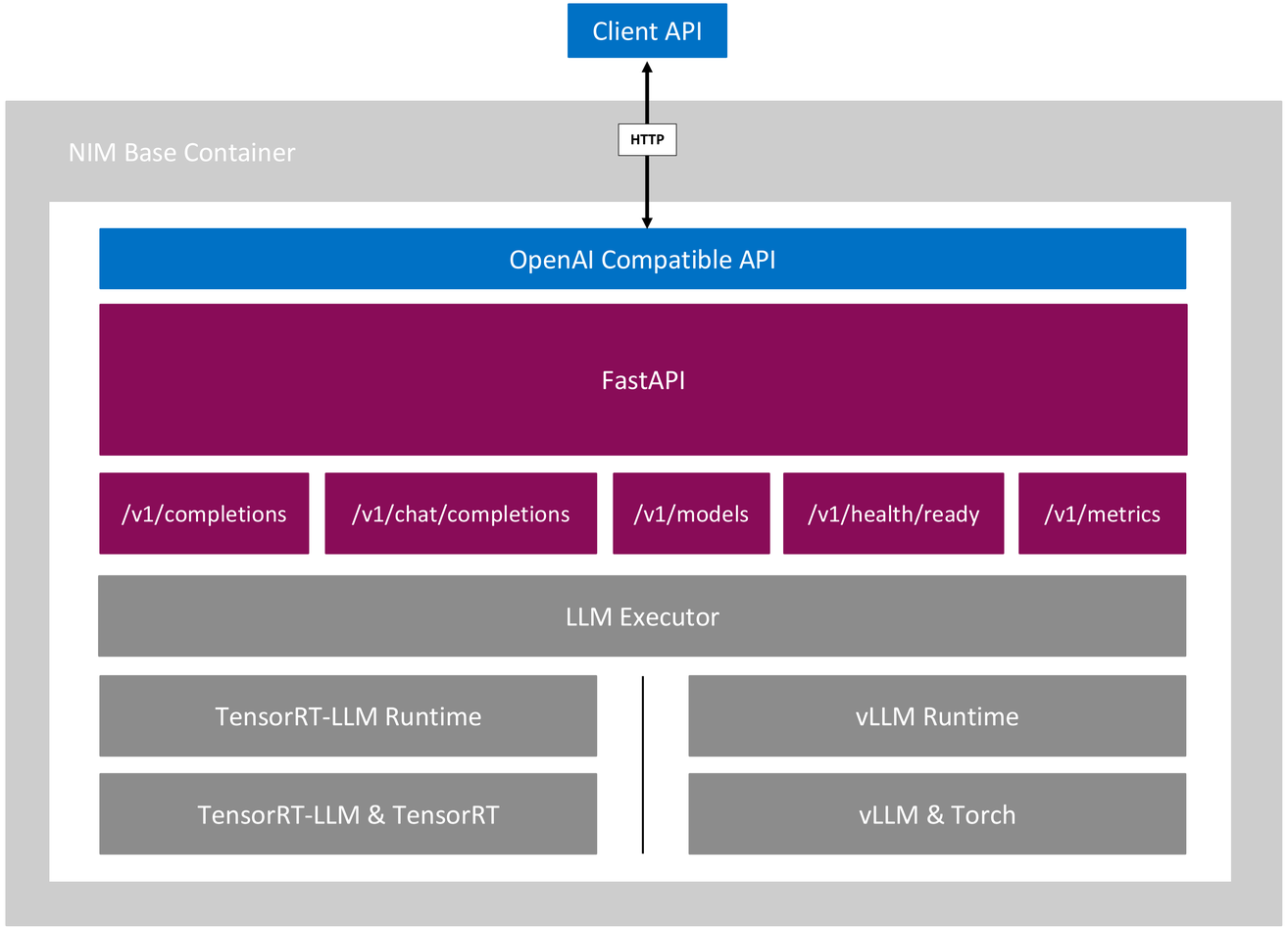
Computer Vision & Simulation

Safety

build.nvidia.com

NVIDIA NIM for LLM Architecture

- HTTP REST API conforms to OpenAI specification for easy developer integration
- Liveness, health check and metrics endpoints for monitoring and enterprise management
- NVIDIA NIM includes multiple LLM runtimes
  - TensorRT-LLM and vLLM
  - Runtime is selected based on detected hardware and available optimized engines, with preference given to optimized engines



Locally Built TensorRT-LLM Inference Engines

Seamlessly deploy SFT models and ensure TensorRT-LLM optimized performance on any NVIDIA GPU

Deploy NIM with custom model weights with a single command

NIM automatically builds and loads an optimized TensorRT-LLM inference engine and deploys the fine-tuned model for inference.

```
export CUSTOM_WEIGHTS=/path/to/customized/llama
docker run -it --rm --name=llama3-8b-instruct \
    -e NIM_FT_MODEL=$CUSTOM_WEIGHTS \
    -e NIM_SERVED_MODEL_NAME="llama3.1-8b-my-domain" \
    -e NIM_CUSTOM_MODEL_NAME=custom_1 \ # set this to cac
    -v $CUSTOM_WEIGHTS:$CUSTOM_WEIGHTS \
    -u $(id -u) \
    $NIM_IMAGE
```

Optionally, use `list-model-profiles` and `e NIM_MODEL_PROFILE=<profile_name>` to list and specify alternative prebuilt and locally buildable, optimized engine profiles for single command deployment

Specify tensor parallelism for N GPUs, or latency versus throughput optimization preference (supported GPUs only).

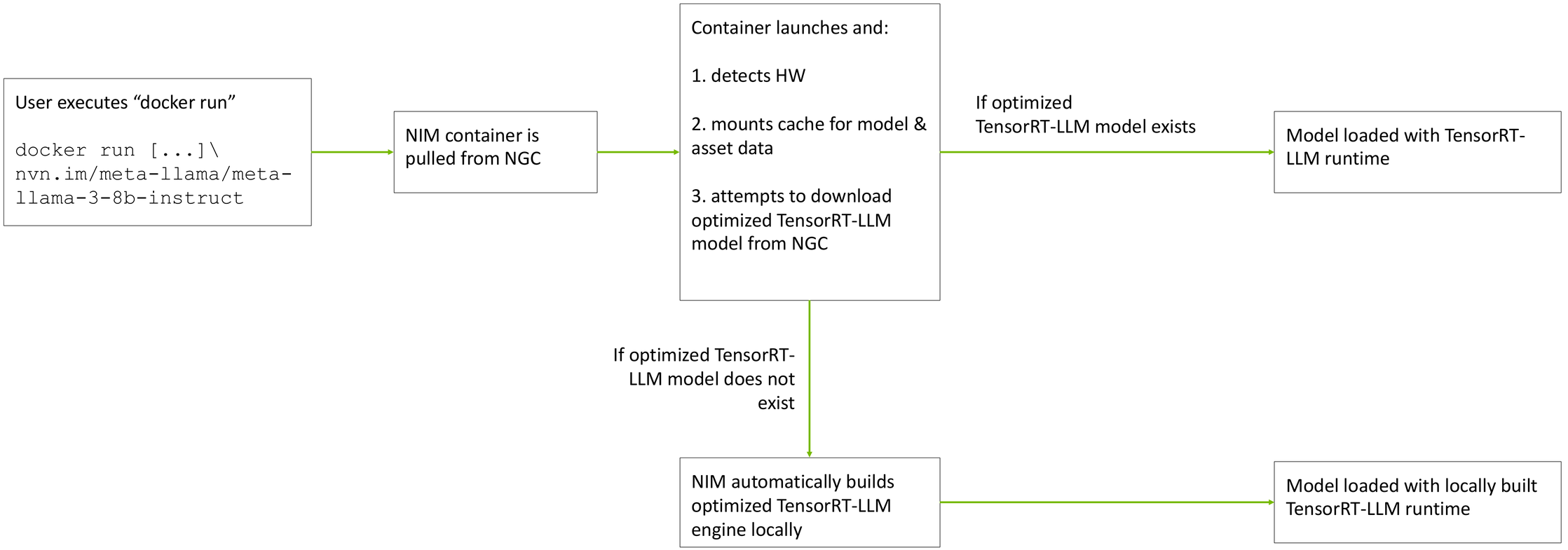
Locally built optimized engines ensure TensorRT-LLM optimized performance on *any* NVIDIA GPU

If a prebuilt, TensorRT-LLM optimized inference engine is not available at NIM launch, NIM automatically builds one in the local environment, loads it and deploys the model through the same single-command deployment process.

```
docker run -it --rm --name=meta-llama3-8b-instruct \
    --runtime=nvidia \
    --gpus all \
    --shm-size=16GB \
    -e NGC_API_KEY \
    -v ~/.cache:/opt/nim/.cache \
    -u $(id -u) \
    -p 8000:8000 \
    nvcr.io/nvidian/nim-llm-dev/meta-llama3-8b-instruct
```

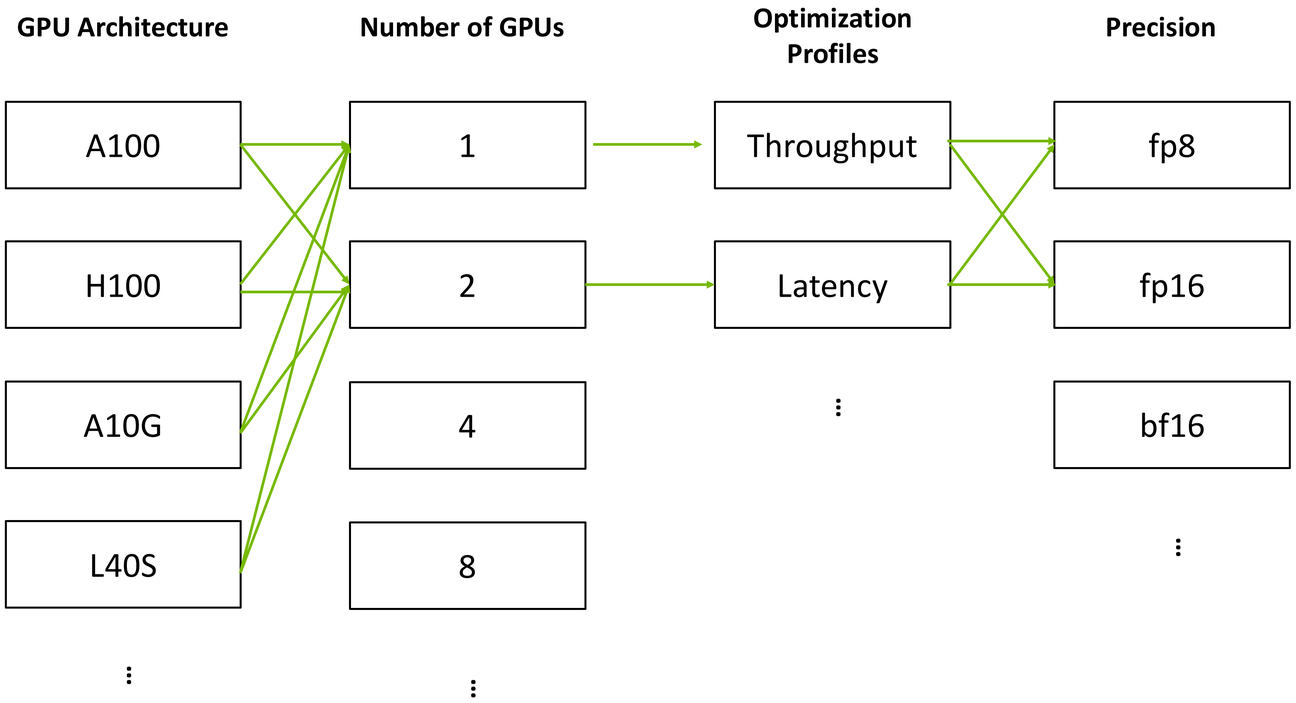
Same single-command deploy

NVIDIA LLM NIM Pull Sequence



How NIMs Leverage Optimized Engines

Llama-3-8B-Instruct Optimization for NVIDIA GPUs with LoRA Support Option



Optimized Models

llama3-8b-instruct:0.10.0+cbc614f5-**h100**x1-fp8-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**h100**x2-fp8-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**h100**x1-fp16-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**h100**x1-fp16-**lora**

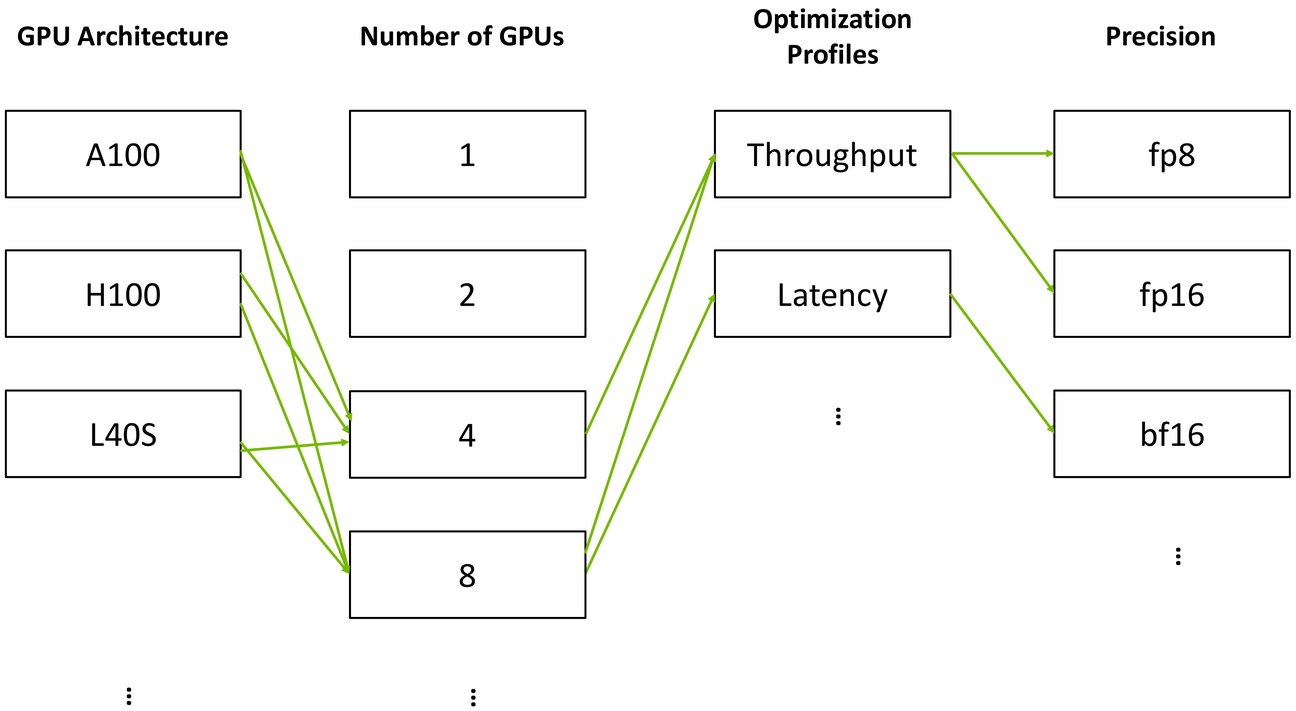
llama3-8b-instruct:0.10.0+cbc614f5-**a100**x1-fp16-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**a100**x1-fp16-**lora**
llama3-8b-instruct:0.10.0+cbc614f5-**a100**x2-fp16-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**h100**x2-fp16-**latency**

llama3-8b-instruct:0.10.0+cbc614f5-**a10g**x2-fp16-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**a10g**x1-fp16-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**a10g**x1-fp16-**lora**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x2-fp8-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x2-fp16-**latency**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x1-fp8-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x1-fp16-**throughput**
llama3-8b-instruct:0.10.0+cbc614f5-**l40s**x1-fp16-**lora**

```
Detected 3 compatible profiles.
Valid profile: 17190a87573ad181e4a55b96d1a84b52f1c82c6542e2404473b25011ebfb403e (tensorrt_llm-A100-bf16-tp1-balanced)
Valid profile: 4ca08b7e7f1d59d5da5f761969320738d6922f25f26f334ab344c21f2e57348f (tensorrt_llm-A100-fp16-tp1-throughput)
Valid profile: 15fc17f889b55aedaccb1869bfeadd6cb540ab26a36c4a7056d0f7a983bb114f (vllm)
Selected profile: 17190a87573ad181e4a55b96d1a84b52f1c82c6542e2404473b25011ebfb403e (tensorrt_llm-A100-bf16-tp1-balanced)
Profile metadata: llm_engine: tensorrt-llm
```

How NIMs Leverage Optimized Engines

Llama-3-70B-Instruct Optimization for NVIDIA GPUs with LoRA Support Option



Optimized Models

lllama3-70b-instruct:0.10.0+cbc614f5-**a100**x8-bf16-**latency**
llama3-70b-instruct:0.10.0+cbc614f5-**a100**x4-fp16-**throughput**
llama3-70b-instruct:0.10.0+cbc614f5-**a100**x4-fp16-**lora**

llama3-70b-instruct:0.10.0+cbc614f5-**h100**x8-fp8-**latency**
llama3-70b-instruct:0.10.0+cbc614f5-**h100**x8-fp16-**latency**
llama3-70b-instruct:0.10.0+cbc614f5-**h100**x4-fp8-**throughput**
llama3-70b-instruct:0.10.0+cbc614f5-**h100**x4-fp16-**throughput**
llama3-70b-instruct:0.10.0+cbc614f5-**h100**x4-fp16-**lora**

llama3-70b-instruct:0.10.0+cbc614f5-**l40s**x8-fp16-**throughput**
llama3-70b-instruct:0.10.0+cbc614f5-**l40s**x8-fp8-**latency**
llama3-70b-instruct:0.10.0+cbc614f5-**l40s**x4-fp8-**throughput**
llama3-70b-instruct:0.10.0+cbc614f5-**l40s**x8-fp16-**lora**

Enterprise-ready Inference for a World of LLMs

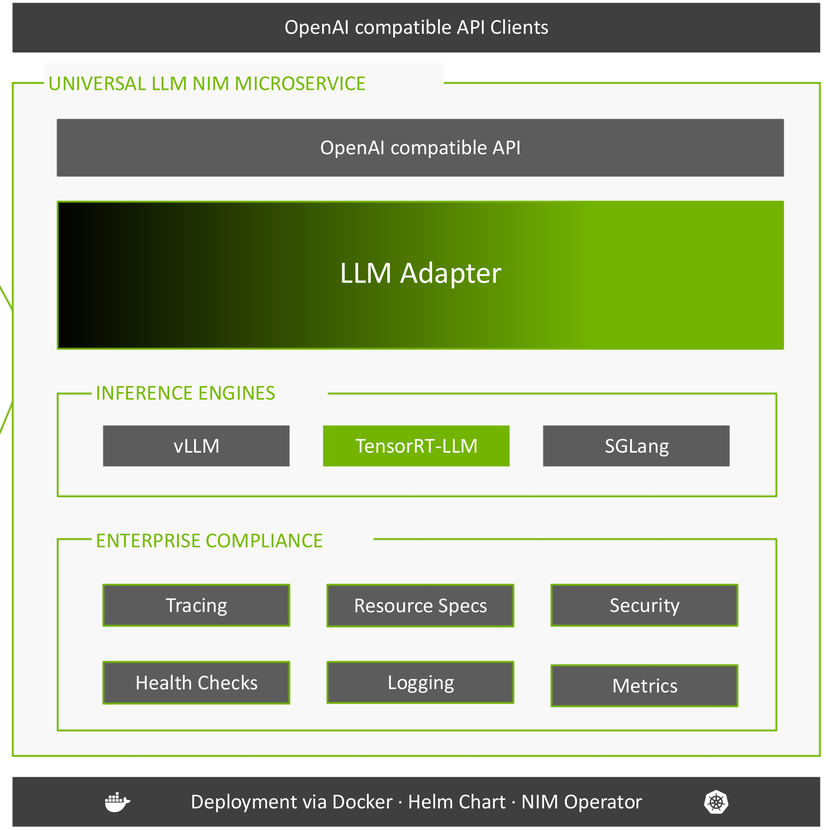
Single NIM container with multiple inference backends for rapid, reliable deployment of a broad range of LLMs

100K+

public and private LLMs

```
> Docker run universal-llm-nim [your_LLM]
```

vLLM NVIDIA SGL



推理運算需求呈指數級增長

推動力來自推理模型與 AI Agent



更大的模型

上千億的模型參數



更長的思考時間

思考 token 數提升 100 倍



更長的上下文

輸入 token 可達百萬級

NVIDIA Dynamo

最佳化的大規模 AI 模型推論軟體

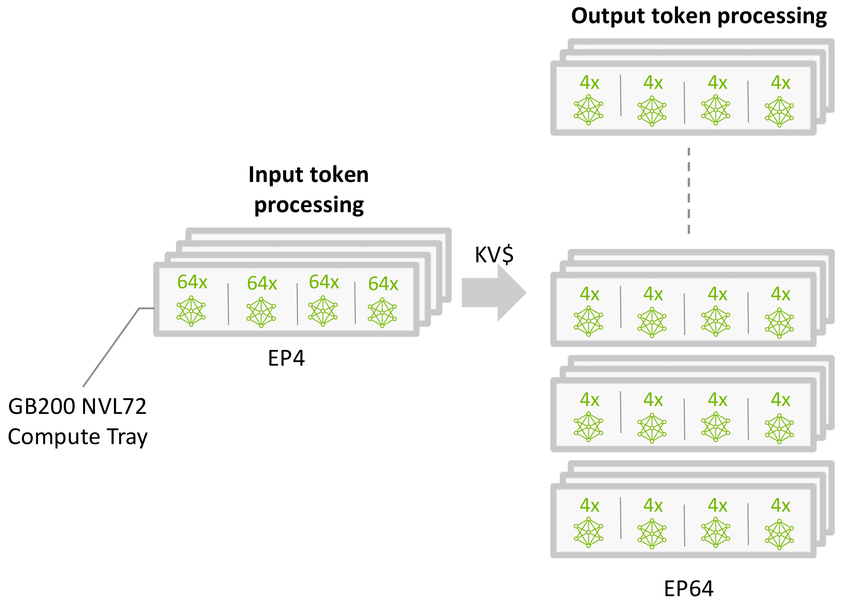
30X
AI Factory Throughput & Revenue
Large models
on Blackwell

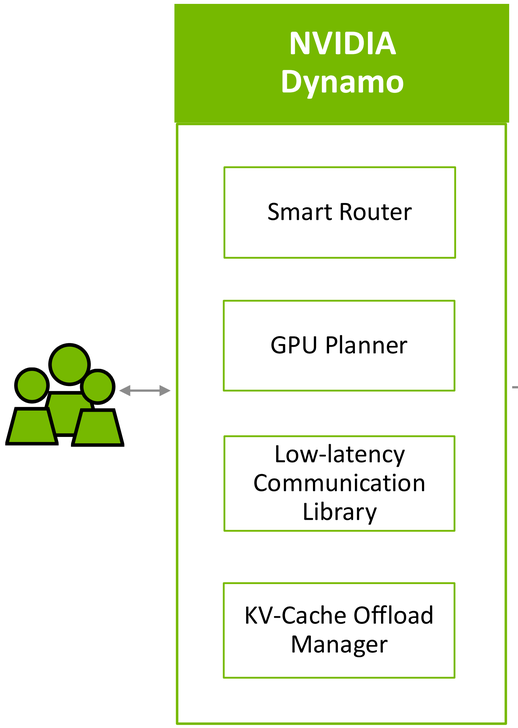
1000+
GPU Scale for a single query

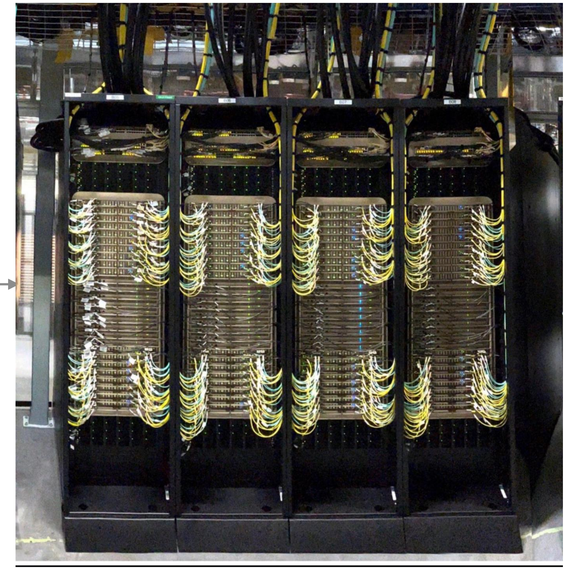
2X
Throughput & Revenue
Llama Models
On Hopper

Distributed and Disaggregated Serving

Reasoning Model



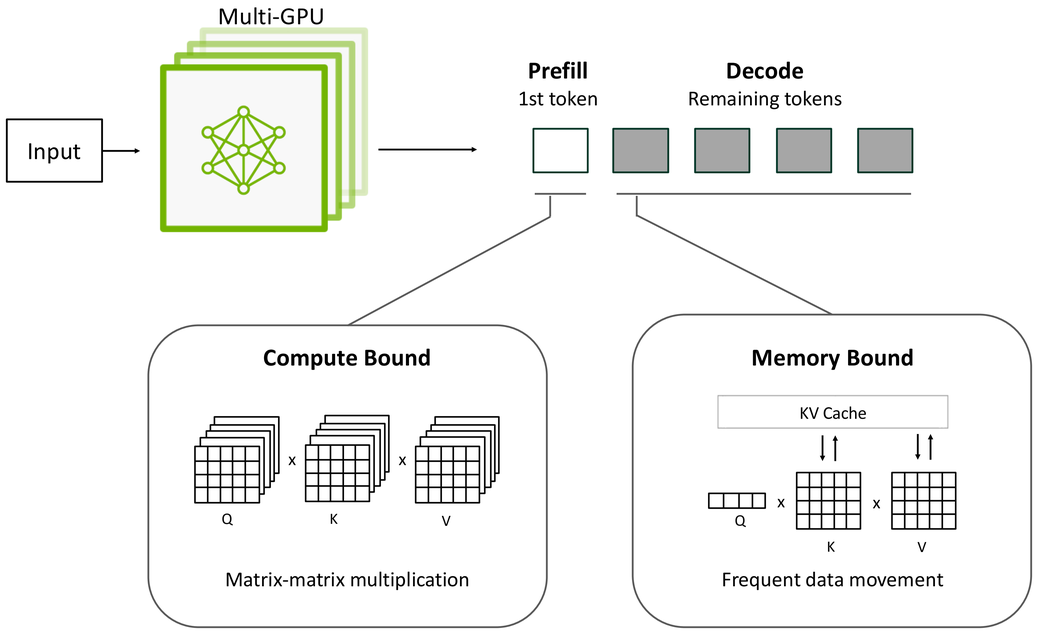




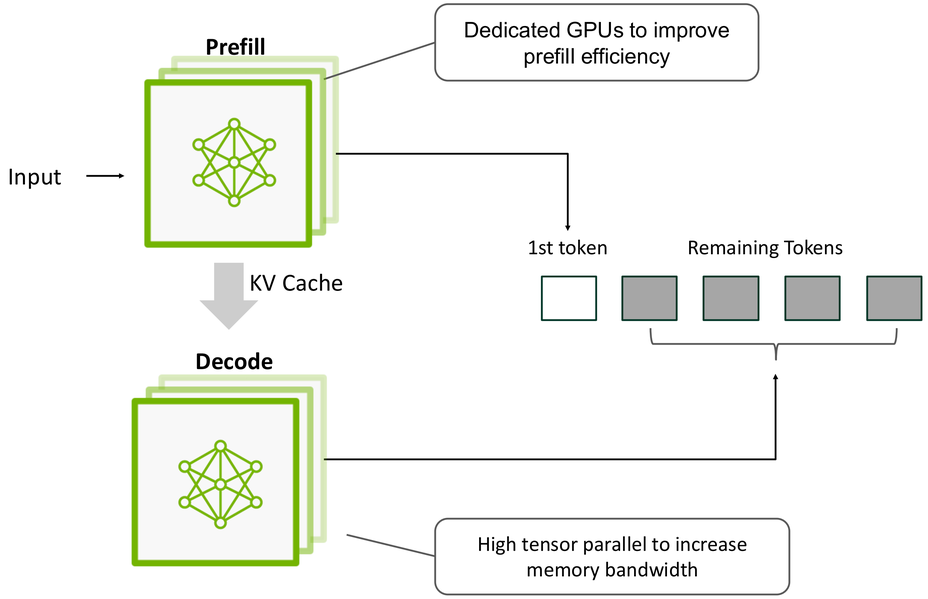
全新推理最佳化技術，全面提升推理效能

解耦式服務將預填充與解碼分離，實現各自獨立優化

傳統的模型推理方式



解耦式模型推理方式



更有彈性地優化模型推理成本與使用者體驗

在明天上課之前，請先到Huggingface取得模型授權

Llama-3.1-8B-Instruct

Hugging Face | Search models, datasets, users... | Models | Datasets | Spaces | Community | Docs | Pricing | Log In | Sign Up

meta-llama / **Llama-3.1-8B-Instruct** | like 4.11k | Follow Meta Llama 48.3k

Text Generation | Transformers | Safetensors | PyTorch | 8 languages | llama | facebook | meta | llama-3 | conversational | text-generation-inference

arxiv:2204.05149 | License: llama3.1

Model card | Files and versions xet | Community 235 | Train | Deploy | Use this model

You need to agree to share your contact information to access this model

The information you provide will be collected, stored, processed and shared in accordance with the Meta Privacy Policy.

LLAMA 3.1 COMMUNITY LICENSE AGREEMENT

Llama 3.1 Version Release Date: July 23, 2024

"Agreement" means the terms and conditions for use, reproduction, distribution and modification of the Llama Materials set forth herein.

"Documentation" means the specifications, manuals and documentation accompanying Llama 3.1 distributed by Meta at https://llama.meta.com/doc/overview....

Log in or Sign Up to review the conditions and access this model content.

Downloads last month
5,386,403



Safetensors

Model size 8.03B params | Tensor type BF16 | Chat template | Files info

Inference Providers NEW | Fireworks +4

Text Generation | Examples

Input a message to start chatting with **meta-llama/Llama-3.1-8B-Instruct**.

NVIDIA

